

國立中正大學資訊工程研究所

碩士論文

使用軟體再生與系統層級冗餘技術提升系統可用率之
研究

**Using Software Rejuvenation and System-Level
Redundancy for Enhancing System Availability**

指導教授 陳鵬升 博士

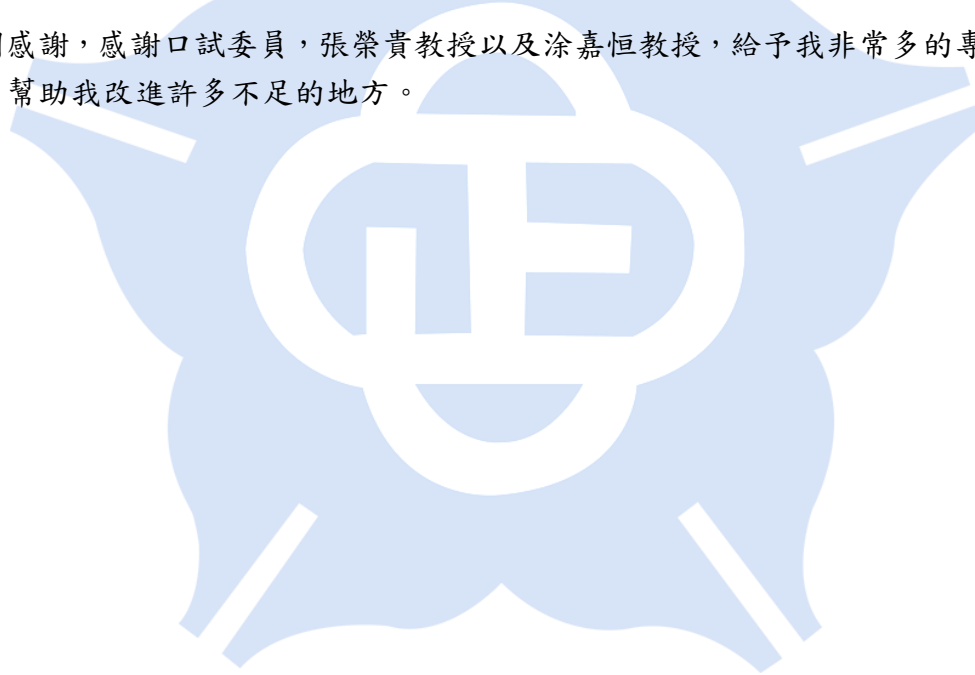
研究生 蔡曜隆

中華民國一百一拾年六月

致謝

我非常感謝我的各位同學，柏維陪伴我打薩爾達傳說，這些點點滴滴，我終生難忘。柏豪是揪團高手，帶我各處玩，我都不用擔心迷路。新佑是我的 PUBG 大師傅，教導我狙擊的深厚功力。旻禧是全實驗是最有女人味的同學，可以在她旁邊長達兩年時間，我此生無憾。最後感謝鵬升老師的兩年多年的尊尊教誨，讓我獲益良多。同時感謝學長姊的幫忙才可以讓我的碩士生活非常有趣，也感謝一路上幫助過我的大家。

特別感謝，感謝口試委員，張榮貴教授以及涂嘉恒教授，給予我非常多的專業建議，幫助我改進許多不足的地方。



摘要

隨著工業科技的迅速發展，越來越多自動化設備與一起我們生活(例如:自動駕駛汽車、醫療設備、無人機、物聯網家電)。這些產品都需要高穩定性和高可靠度。他們如果發生故障可能造成人類的生命危險。這些故障也可能來自軟體的老化。因此，開發提升系統可靠性以及降低軟體老化的技術是一個重要議題。

本論文主要設計與實現在嵌入式系統中，整體架構是由一個主系統和一個冗餘系統組成，以及軟體再生工具做記憶體用量監測。透過 socket TCP/IP 連接，結合心跳機制檢察系統是否發生故障。透過執行角色的轉換完成接手目標程式。利用 checkpointing tool(DMTCP)建立系統快照，為了從故障中恢復。並且結合 GlusterFS 提供系統一個可靠的儲存環境。最後整合軟體再生工具，如果記憶體用量超過門閥值則啟動軟體再生。透過 benchmark 量測 和測試情況，可以有效提高容錯能力和降低軟體老化造成的影響，並且能在故障後恢復程式狀態。

關鍵字：System-level redundancy, Reliability, Fault tolerance, Software aging, Software rejuvenation

Abstract

With the rapid development of industrial technology, more and more automated devices live with us (for example: self-driving cars, medical equipment, drones, Internet of Things appliances). These products require high stability and high reliability. If they fail, they may endanger human life. These failures may also come from the aging of the software. Therefore, the development of technologies to improve system reliability and reduce software aging is an important issue.

This paper is mainly designed and implemented in embedded systems. The overall architecture is composed of a primary system and a redundant system, as well as software rejuvenation tools for memory usage monitoring. The two systems are connected via socket TCP/IP. Combine the heartbeat mechanism to check whether the system is malfunctioning, and through the role exchange of implementation to complete the takeover of the target program. Use the checkpointing tool (DMTCP) to create a system snapshot in order to recover from a failure. Moreover, combine with GlusterFS to provide a reliable storage environment for the system. Finally, the software rejuvenation tool is integrated, and if the memory usage exceeds the threshold, the software rejuvenation is started. Through benchmark measurement and test conditions, fault tolerance can be effectively improved and the impact of software aging can be reduced, and the program state can be restored after a failure.

Keywords: System-level redundancy, Reliability, Fault tolerance, Software aging, Software rejuvenation

Contents

Chapter 1	Introduction.....	1
1.1.	Related Works	2
1.2.	Contribution	5
1.3.	Organization	5
Chapter 2	Background.....	6
2.1.	Checkpointing tool DMTCP.....	6
2.2.	Distributed file system GlusterFS.....	7
2.3.	Memory measurement tool stress-ng.....	8
2.4.	Memory tracking tool GNU glibc	8
2.5.	Software Rejuvenation Agent (SRA)	9
Chapter 3	Design and Implementation	10
3.1.	Communication mechanism	13
3.2.	Checkpointing	20
3.3.	Dependable storage	21
3.4.	Software Rejuvenation Agent (SRA)	23
3.5.	Assistant tool	27
3.6.	mtrace hook operation	31
Chapter 4	Experiment.....	38
4.1.	Environment configuration.....	38
4.2.	Experimental results	40
4.3.	Experimental data of mtrace.....	53
Chapter 5	Conclusion	58

List of Figures

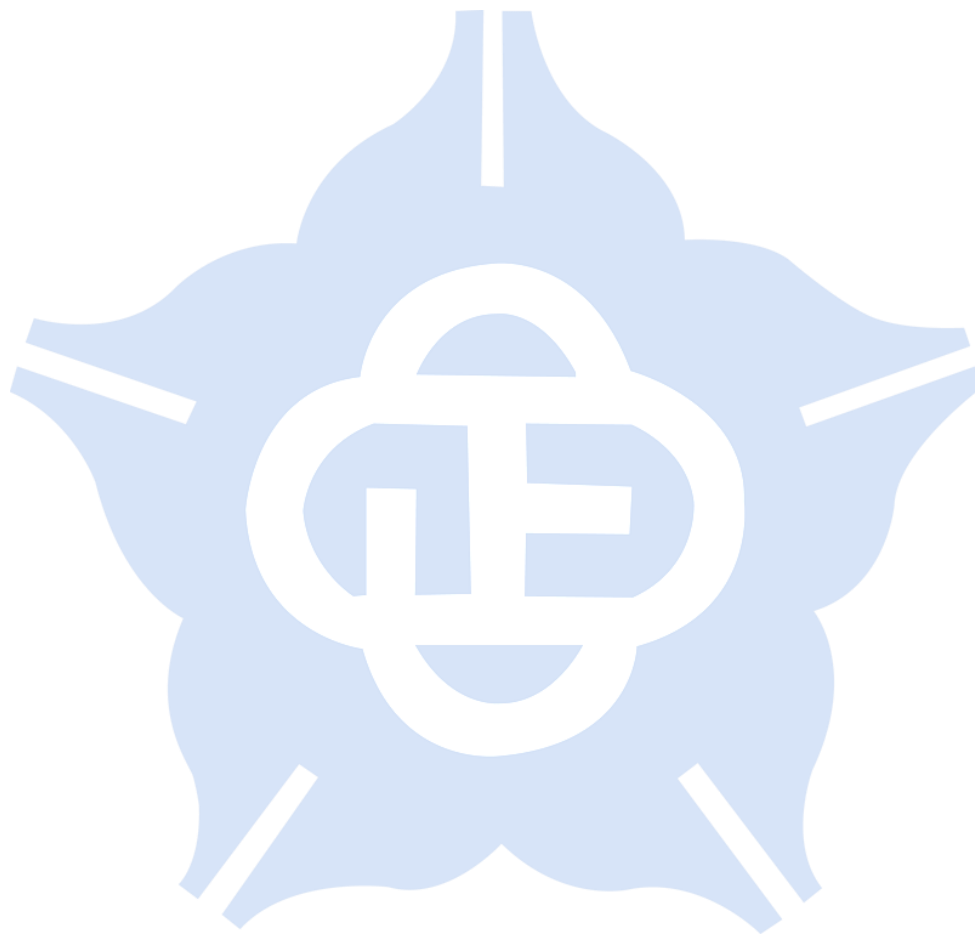
Figure 1.1: Concept of the proposed system-level redundancy.	2
Figure 2.1: A simple two-node cluster file system.	7
Figure 3.1: Crash of the primary system (part 1).	11
Figure 3.2: Crash of the primary system (part 2).	12
Figure 3.3: Crash of the redundant system.	12
Figure 3.4: Execution flow between the primary and redundant systems. ..	14
Figure 3.5: Contents of the auto-start configuration file.	16
Figure 3.6: Example of role exchange.	18
Figure 3.7: Command-line options used in DMTCP.	20
Figure 3.8: Command-line options for the DMTCP resume program.	20
Figure 3.9: Command-line options used to configure GlusterFS	21
Figure 3.10: Content of the glusterd.vol config file.	22
Figure 3.11: Flow chart of memory limit measurement.	24
Figure 3.12: Software rejuvenation execution flow.	26
Figure 3.13: Screenshot of the memory measurement GUI.	27
Figure 3.14: Screenshot of the fault-tolerant service GUI.	29
Figure 3.15: Screenshot of setting static IP.	29
Figure 3.16: Enter all the parameters.	30
Figure 3.17: Introduction to glibc installation.	31
Figure 3.18: mtrace program snippet(hook.c:304).	32
Figure 3.19: Program snippet of tr_mallochook(hook.c:186)	33
Figure 3.20: Program snippet of tr_where(hook.c:77).	34
Figure 3.21: Program snippet of muntrace(hook.c:353).	35

Figure 3.22: Flow chart of the user calling malloc.	36
Figure 3.23: Program snippet of malloc_hook_ini(hook.c:28)....	36
Figure 3.24: Filter program fragments of user programs.....	37
Figure 4.1: Screenshot on the primary system.....	40
Figure 4.2: Screenshot on the redundant system.	41
Figure 4.3: Screenshot on the primary system during the redundant system crash.	41
Figure 4.4: Primary system executes the target program.....	42
Figure 4.5: Redundant system operation monitor.....	42
Figure 4.6: Redundant system detected primary system failure.	43
Figure 4.7: Redundant system executes the target program.	43
Figure 4.8: Primary system becomes a monitor.....	43
Figure 4.9: Software Rejuvenation running screenshot.....	44
Figure 4.10: Screenshot of software rejuvenation trigger.	44
Figure 4.11: Example with memory leak and double-free errors.	56
Figure 4.12: Screenshot of DMTCP memory tracking.....	57
Figure 4.13: Screenshot of memory tracking of general programs.	57
Figure 4.14: Evaluation result of glibc.sh memory test.	57

Lists of Tables

Table 4-1: Raspberry Pi-related parameters.	38
Table 4-2: Benchmark programs.	39
Table 4-3: Benchmark test (checkpoint time = 5), size of executable code =8256 bytes.	46
Table 4-4: Benchmark test (checkpoint time = 10), size of executable code =8256 bytes.	46
Table 4-5: Benchmark basicmath_large (checkpoint time = 5), size of executable code = 12656 bytes.	47
Table 4-6: Benchmark basicmath_large (checkpoint time = 10), size of executable code = 12656 bytes.	47
Table 4-7: Benchmark basicmath_small (checkpoint time = 5), size of executable code = 8560 bytes.	48
Table 4-8: Benchmark basicmath_small (checkpoint time = 10), size of executable code = 8560 bytes.	48
Table 4-9: Benchmark isqrt (checkpoint time = 5), size of executable code= 8208 bytes.	49
Table 4-10: Benchmark isqrt (checkpoint time = 10), size of executable code= 8208 bytes.	49
Table 4-11: Benchmark rad2deg (checkpoint time = 5), size of executable code = 8336 bytes.	50
Table 4-12: Benchmark rad2deg (checkpoint time = 10), size of executable code = 8336 bytes.	50
Table 4-13: Benchmark matrix2 (checkpoint time = 5), size of executable code = 8344 bytes.	51
Table 4-14: Benchmark matrix2 (checkpoint time = 10), size of executable code = 8344 bytes.	51

Table 4-15: Restart time of the SRA.....	52
Table 4-16:CWE ID.....	54
Table 4-17: Benchmark programs for measuring overheads.....	55



Chapter 1 Introduction

The complex computer system is closely integrated with our daily life, including general personal assistants, medical systems, industrial control systems, aviation systems, self-driving cars, and robots. Some unmanned equipment requires high reliability and stability in normal operation, and their failure can lead to serious problems or even threaten lives[1][2]. Therefore, the development of techniques to enhance system reliability is an important issue [3][4][5][6]. System-level redundancy uses multiple redundant systems to tolerate failures[7]. If the primary system fails, the redundant system can take control to resume normal operation.

Our system is especially aimed at the embedded system, because the memory space of the embedded system is usually very limited. So we designed Software Rejuvenation Agent (SRA) to do real-time memory monitoring to achieve the protective effect. Furthermore the checkpoint tool and Glusterfs we use both have the characteristics of lower overheads, reducing the load burden of the embedded system. Finally, we choose mtrace to help us output memory-related information, because it uses very low memory space and low average running time, so mtrace running on the embedded system is a very big advantage.

A system-level redundancy technique is designed and implemented here for embedded systems. The whole construct consists of primary and redundant systems connected using network socket programming APIs. The proposed technique is built on a Raspberry Pi 4 evaluation board [8], because Raspberry Pi 4 has the advantage of being cheap and widely used, rich source code development environment, Figure 1.1. A heartbeat mechanism checks whether the primary system is alive. The redundant system transmits a heartbeat packet to the primary system at regular intervals. If the primary system receives the heartbeat packet, it will respond appropriately. Otherwise, it is assumed to have failed, and then the redundant system will execute processes to take control. At the same time, the primary system will switch to the monitor. That is, the primary system periodically sends a heartbeat packet to the redundant system. This mechanism can greatly reduce the system restart time. In addition, we use the Software Rejuvenation Agent (SRA) to track the memory of the target program, and promptly perform the alarm system and software rejuvenation operations. GlusterFS[9][10], an

open-source distributed file system, is used to consolidate the disk storage resources of two different systems, replicated the data in the file system to ensure high availability. In addition, distributed multi-threaded checkpoint (DMTCP)[11] is used to store the execution state of the target program, which can help us recover from failures. GNU glibc can provide real-time memory monitoring log files to help users track memory leaks and double free. Preliminary experimental results for benchmarking and test situations show that the proposed approach can improve fault tolerance and resume operations after failure. This work provides an easy and systematic approach to add system-level redundancy to improve the fault tolerance of a variety of general systems. The whole system we implemented takes Raspberry Pi as the target machine and introduces the watchdog mechanism on Raspberry Pi. The system can on other embedded systems, slight modifications.

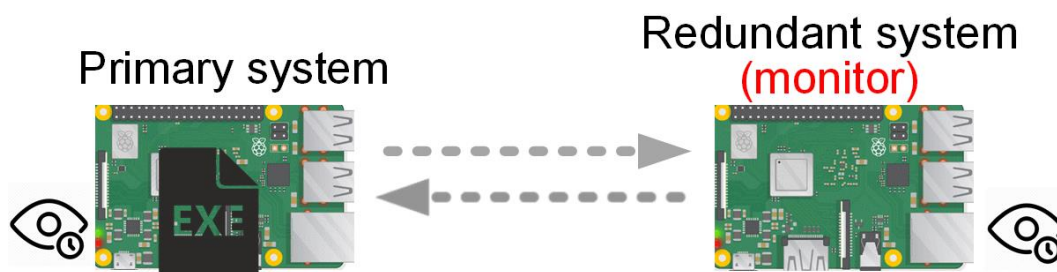


Figure 1.1: Concept of the proposed system-level redundancy.

1.1. Related Works

Among the causes of system crashes caused by various failures, we are particularly specialized in software aging. Now it is known that software failures are the main source of system interruptions. Many studies and a large amount of evidence show that software aging is a common phenomenon. Will cause the state of the system to decline over time.

Software aging is a major symptom of system resource depletion, data corruption, and the accumulation of numerical errors, (e.g., Fatal Error: How Patriot Overlooked a Scud [12] , An empirical investigation of fault types in space mission system software [13]) These two papers detail the aging of the software in an unmanned environment

that caused errors in the calculation data of the Patriot missile and the serious mistakes that killed its own soldiers, as well as the analysis of 520 software failures detected by the aerospace system in 18 missions. The author defines detailed bug classifications. 1. Bohrbug: It is a fault that is easy to isolate. It will appear under a set of well-defined conditions, that is, errors due to human programming. 2. Mandelbug: It is not easy to isolate the fault, and the trigger conditions and effects are very complicated. However, after analyzing 520 software faults, it is found that Bohrbugs accounted for 61.4% and Mandelbugs accounted for 36.5%. The analysis of Mandelbugs is related to the aging time of the software. Some papers have begun to study how to predict system resource exhaustion early. The prediction methods can be divided into model-based ones, measurement-based ones, and gate thresholds ones. The model-based approaches are aimed at building analytic models of system degradation and solving these models for determining the effectiveness of software rejuvenation as well as for deriving optimal rejuvenation schedules[14][15]. However, in-depth understanding and discovery of model-based methods have a major defect that some basic assumptions of these random mathematical models are often too ideal and are not easy to be verified in the engineering field. The basic idea of measurement-based approaches is to directly monitor attributes that are subject to software aging. For example, in a system with memory leaks not all of the memory allocated to a task is necessarily released after its completion, leading to an increasing trend in memory usage[16][17]. Measure-based methods can use machine learning to train the model, and the model learned can also have high accuracy. However, the biggest problem is that the most important foundation of machine learning is dataset. At present, the measuring-based methods all assume a random model or generate related memory leakage data by applying pressure on the system as dataset. The drawback is how similar these datasets are to real software aging data, a question that remains unanswered. Suppose there is a memory leak that is not in the dataset. The result is a poor prediction. Therefore, we adopt the method based on the threshold value [18] to carry out real-time monitoring of the system to be protected. When the memory value reaches or exceeds the user-defined value, the software rejuvenation is made to make the system return to a healthy state. The Software Rejuvenation Agent (SRA) used by the server in IBM [19] can indeed increase the system's execution time by rejuvenating the software-aging program.

Mandelbugs is known to be a type of failure that is not easy to isolate. It can be a sudden failure or a slow accumulation of errors like software aging. If it is a sudden

failure, how to ensure that the system can continue to run and continue to execute from the interrupted program state is a double protection.

Therefore, the system fault tolerance mechanism is very important in many companies. For example, VMware [20] adds fault tolerance features to their virtual machines. They proposed a deterministic-state-machine method to make the two machines have the same initial state, accept the same input, and the same order. The results of the tasks performed by the two machines will be the same. LEOSYS [21] designed a fault-tolerant server. Because VMware is a commercial product, it does not disclose programming details. As a result, the current fault-tolerant system is more expensive, and not all users can afford it. Some schools [22][23] also develop fault-tolerant techniques for virtual machines. Our goal is focused on embedded systems for software rejuvenation services and compute-intensive programs. These applications require high reliability and high stability.

We combined software rejuvenation and system-level redundancy technology to prevent system collapse and continue to execute the target program after system collapse. We know that aging software will cause problems due to memory leaks. However, there is no relevant information about the cause of the memory leak, which is very difficult to debug later. At present, well-known dynamic memory analyzers include Valgrind [24][25], glibc mtrace [26] etc. When the Valgrind tool is started, the program that needs to be analyzed is loaded into the same process space as the Valgrind tool program, and then the dynamic binary recompilation technology of JIT (Just-in-Time) is used to divide the program into small program blocks for recompilation. In the recompilation process, the Valgrind kernel will convert the machine code of the corresponding program block into the intermediate language, and the plug-in will analyze the corresponding code plug-ins in the intermediate language. Finally, the intermediate language representation will be converted into the original machine code through the kernel and executed on the target computer. The Valgrind kernel spends most of its time executing the translation between the above machine code and the intermediate language representation, while none of the original program's machine code is executed. Most of the time of the Valgrind kernel is spent in the mutual translation and execution of the above machine code and intermediate language representation, while all the machine code of the original program is not executed, but is executed after the plug-in. mtrace transfers the program control flow that should

have executed malloc to mtrace program to perform data collection (collect the location and size of the memory section of `free()` and `malloc()`), and makes use of shell script provided by glibc to conduct memory leak analysis. A common drawback of Valgrind and mtrace is that a full memory leak report cannot be made until the program has finished executing. However, many applications never stop executing, which means they never see the results reported. Therefore, we modify the related source code of glibc library to track the memory with DMTCP program.

1.2. Contribution

1. The whole system is composed of a primary system and redundant system, which are connected by network and form a reliable architecture.
2. The Heartbeat transport protocol is used to detect whether a primary system or redundant system is functioning properly.
3. The integration of DMTCP provides the ability to continue execution from the state it was in before the error occurred after a system error response.
4. Integrate GlusterFS to provide reliable storage space.
5. Modify and integrate the GNU glibc library and DMTCP to track memory usage.
6. Automatically track the usage status of specific program memory and instantly output relevant information.
7. Develop a friendly GUI-based assistant tool to assist users to build the whole system, including DMTCP and GlusterFS related Settings, to protect the program, software rejuvenation related parameters, etc.

1.3. Organization

The remainder of this thesis is organized as follows. Chapter 2 describes the software tools and structures for system-level redundancy, as well as the system compression software and memory tracking tools. Chapter 3 describes the realization process in detail. Two system-level redundancy modes and two GUI-based assistant tools are developed. Chapter 4 describes the experimental structure and gives the experimental results. Finally, concluding in Chapter 5

Chapter 2 Background

2.1. Checkpointing tool DMTCP

The purpose of the checkpoint is to create an image (snapshot) of a running program. The snapshot is to record the current state of the program, and the snapshot can be used to restore the program and continue execution from the interrupted location.

Checkpoints have been researched and used for a long time in the High-Performance Computing [27][28]. There are many different kinds of checkpoint tools, which can be divided into application-level, user-level, and system-level. [29] Each of them has different advantages and disadvantages. The checkpoint tool DMTCP is a transparent checkpoint that can be used in a single-point host or distributed computing without modifying the user's code. DMTCP Merit is easy to install free, and open-source software with low overhead and can support a variety of common programming languages or scripted applications in Linux. DMTCP has been in development for over 20 years and is still evolving today. The latest version, 3.0 in 2019, has been referenced many times. Based on the above advantages, we decided to hand over the program state restoration work to DMTCP.

2.2. Distributed file system GlusterFS

GlusterFS has powerful functions for horizontal scalability. It is also an open-source distributed file system. The horizontal expansion architecture of GlusterFS allows users to flexibly increase resources to enhance storage capacity and performance. It uses TCP/IP or InfiniBandRDMA to share users' storage resources. The advantages of GlusterFS are expandability, and high performance, and high availability. It has flexible volume management and easy installation, and it can handle thousands of nodes.

GlusterFS provides users with three different types of volumes: Distributed Glusterfs Volume, Replicated Glusterfs Volume, Striped Glusterfs Volume. Our goal was to improve the reliability of the storage, so we choose to use the Replicated GlusterFS Volume. Replicated Glusterfs Volume mode can copy the same data to different bricks. Its advantage is that if one of the bricks is damaged, the related data will not disappear or be damaged, which improves the reliability and data sharing of data redundancy. Figure 2.1 is a simple presentation. When running in Replicated GlusterFS Volume mode, if there are different files to be put into the brick, it will copy a file to another brick at the same time.

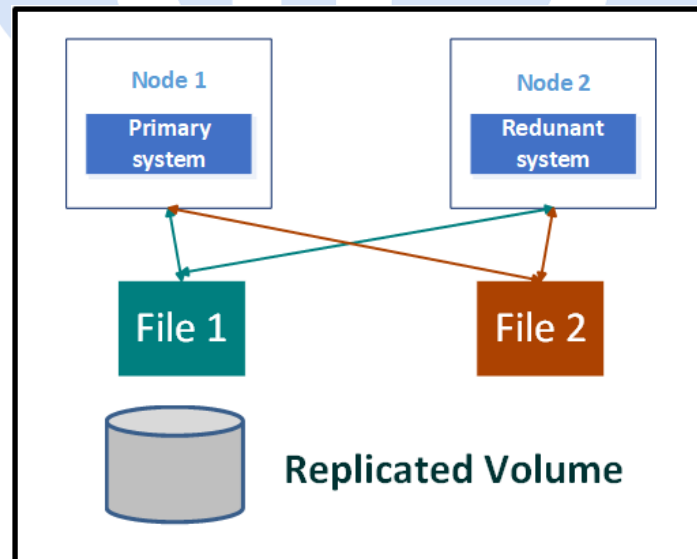


Figure 2.1: A simple two-node cluster file system.

2.3. Memory measurement tool stress-ng

stress-ng [30] has a variety of ways to stress test computer systems. Its purpose is to test the pressure tolerance of various physical subsystems of the computer and various operating system kernel interfaces. stress-ng also has powerful CPU-specific stress tests that can be used for floating point, integer, bit operations, and control flow.

stress-ng[31] also has the following features: 1. Exercises a wide range of system calls, /dev, /sys, /proc interfaces 2. Used by researchers for stress testing 3. System burn-in tests 4. Real Time / Low-latency cyclic measurements and other functions. It can be known that stress-ng is commonly used for system aging and system testing, and stress-ng has a wealth of stress test sources. According to statistics, stress-ng has more than 220 pressure measurement options. It contains CPU cache (icache, dcache) , CPU compute (integer, float, string, searching...), Memory (throughput, VM, RAM tests, paging, stack, brk, and mmap), etc. Because of stress-ng's rich and diverse test environment and free open source code, we chose it as a memory pressure measurement tool.

2.4. Memory tracking tool GNU glibc

glibc is the underlayer system API in the Linux system (including ISO C11, POSIX.1-2008, BSD, operating system-specific API, etc.) Almost any Linux operation will call the glibc implementation and also encapsulate Linux-related instructions. glibc is also an implementation database of the C language. Moreover, the glibc library is free and open-source software. Therefore, it can be modified very much[26]. GNU glibc started around 1988 and has a history of 33 years. Glibc includes almost all UNIX operating standards, so the architecture is very large. The mtrace library in the glibc library source code can provide users with `mtrace()` in the program to enable memory usage tracking of `malloc()` and `free()`. In addition, `muntrace()` turns off tracking the use of `malloc()` and `free()`. A log file will be generated after the program is executed. The log file can use the shell script named `mtrace` provided by the glibc library for memory leak analysis. We achieved this by modifying the mtrace library: combining the DMTCP program to track memory, automatically tracking

specific programs, the real-time output of diary files, analysis of double free, memory leak services, and memory leak size sorting.

2.5. Software Rejuvenation Agent (SRA)

Software Rejuvenation Agent (SRA)[32] is used to monitor consumable resources. If the real-time monitoring value exceeds the user-defined value, SRA will raise system-related alarms or directly perform Software Rejuvenation actions. Software Rejuvenation is to restart the program that is about to crash. Real-time Software Rejuvenation can effectively reduce the system exchange time caused by fault-tolerant services, and achieve the effect of reducing the exchange time. We first use stress-ng to stress test the system to get the real limit value of the system memory. Among them, Stress Its purpose is mainly to test the memory and core stress test of the computer. We use the forced memory stress test method to obtain the limit value of the memory. The user can refer to the tested memory limit value to define the upper limit of protection. stress-ng has many ways to stress test the computer system. After the setting is complete, the SRA combines the top command to monitor the memory usage in real time. We also provide a user interface for the simple setting of related parameter values, which greatly improves user applicability.

Chapter 3 Design and Implementation

Our proposed system-level redundancy includes a primary system and a redundant system. The two systems communicate through a network socket programming API connection. We use a heartbeat mechanism between these two systems to check whether there is a system crash. The redundant system can send heartbeat packets to the primary system periodically to check whether the primary system is running normally. If the primary system sends a heartbeat packet to the redundant system, it means that the primary system is running normally. On the contrary, if the primary system does not send a heartbeat packet to the redundant system, it means that the primary system has crashed, so the redundant system can take over the operation of the target program. After the primary system is restarted and connected to the redundant system, the primary system can continue to be a monitor. The above description shows that the primary system has a sudden failure when it is executing the target program, and the target program is transferred to the redundant system. When the redundant system fails while executing the target program, the process is the same to complete the entire fault-tolerant service.

We provide both the primary system and the redundant system to install a Software Rejuvenation Agent (SRA). The SRA monitors the memory usage of the programs that the user wants to protect. If the memory monitoring reaches or exceeds the user-defined upper limit, As far as software rejuvenation is performed, the protection system can continue to run on the original host, reducing the number of system exchanges, and achieving the shortest recovery time but still providing fault-tolerant services.

We provide two execution modes. The first mode is that the target program is of repetitive execution type, such as temperature sensor and data collection program. These programs do not need to use DMTCP to record the complete program status. Therefore, we only need to ensure that the program can continue to run. Instead of restarting the program state, in the first mode, we will not use DMTCP and GlusterFS in our implementation, so the user only needs to put the target program on the two Raspberry Pis. When the redundant system detects the main When the system crashes, it will start to take over the execution of the program. When the primary system detects a redundant system crash, the process is the same to ensure that the program can continue to run.

Usually, many practical applications will use the second mode. The system needs to store the running state. When the redundant system takes over control, it will resume execution from the storage state, or restore the program state when the fault-tolerant service is exchanged. In the second mode, we will need DMTCP to create program snapshots and cooperate with GlusterFS for reliable sharing of program snapshots.

The proposed system-level redundancy technique consists of two systems: a primary system and a redundant system. The primary system executes a target program that we want to protect. The two systems are connected via a network. A heartbeat mechanism is adopted to monitor whether the primary and redundant systems are working normally. Figure 3.1 and Figure 3.2 show the situation of the primary system crashing. When the primary system crashes (marked as 1), this is detected by the redundant system via the heartbeat mechanism (marked as 2). The redundant system then takes over control and changes its role to the primary system in order to resume the work of that system (marked as 3 and 4). The original primary system automatically restarts by invoking the Raspberry Pi watchdog function (marked as 5). It reconnects to the original redundant system, discovers that the target program is running, and then changes its role to monitor the redundant system (marked as 6). The complete system now works in the same way as depicted in Figure 1.1, but where the roles of the two systems have been reversed. Figure 3.3 shows the situation of the redundant system crashing. After crashing, the redundant system is automatically restarted by the watchdog function, and it then rejoins the complete system to continue monitoring the primary system again.

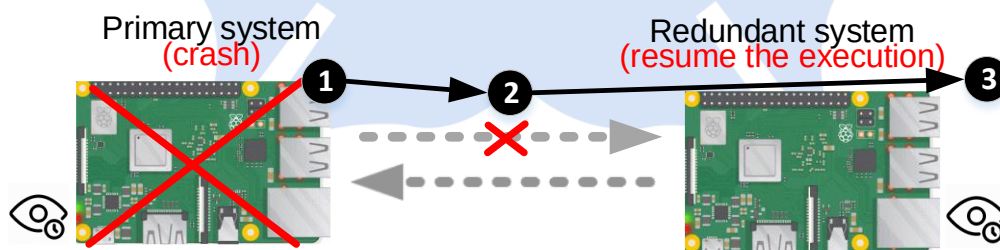


Figure 3.1: Crash of the primary system (part 1).

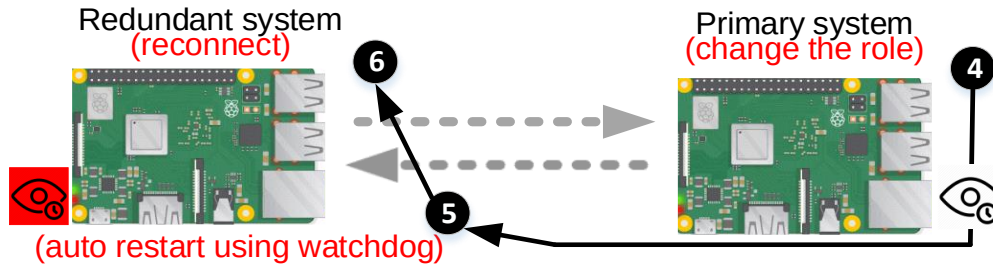


Figure 3.2: Crash of the primary system (part 2).

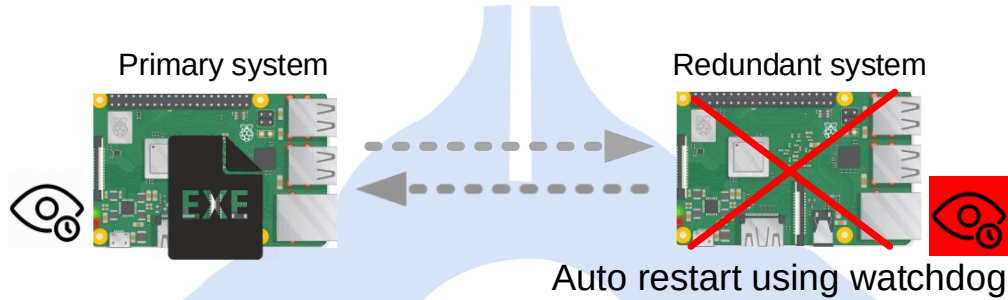


Figure 3.3: Crash of the redundant system.

In addition to system transfer due to system crashes, we also provide a real-time memory monitoring system. When the primary system or redundant system takes over the execution of the target program, the SRA will be activated for memory monitoring, which can prevent the target program from not being executed, But there is more overhead. If the SRA detects that the memory exceeds the user setting value when the primary system or the redundant system is running the target program, the primary system or the redundant system will be forced to restart the target program by the SRA.

Finally, we developed two GUL-based assistant tool. The first tool can automatically complete the tedious software installation and environment configuration, and the second tool can automatically test the memory limit value. These two tools simply input some relevant information fields. When the input is completed, and then click the start button, it can set up and execute the system we proposed.

Our implementation method is divided into six main parts. The first part is the communication algorithm between the primary system and the redundant system. In order to notify each other when a failure occurs on one side, and explain the socket programming code, here will be divided into two parts. The primary system and the redundant system are restarted due to a crash, and the supervisors of the primary system

and the redundant system are switched. The second part explains that checkpoints regularly establish images (snapshots) of running programs in the primary system and redundant systems, in order to restore the execution status before the failure after a failure. The third part explains the establishment of a two-node cluster file system in order to improve the storage reliability of checkpoint images. The fourth part explains the implementation of the software rejuvenation tool operation. The fifth part explains the use of assistant tool to help users build a system-level redundant system. The last part explains how to modify glibc's mtrace to track the memory usage details of the target program for us, which can help users debug.

3.1. Communication mechanism

The communication protocol between the two Raspberry Pi is based on the TCP/IP protocol, and the communication architecture uses client/server architecture, the primary system is the client, and the redundant system is the server. The two parties have established a two-way communication channel. After the system is successfully connected, the parent process is responsible for processing heartbeat data transmission, and a child process is forked to execute the target program, and the redundant system will fork a child process to process the heartbeat message reception and response messages.

Figure 3.4 shows the connection between the two systems and the process of processing heartbeat packets. The following subsections will describe the details of the operation process of the two systems in detail.

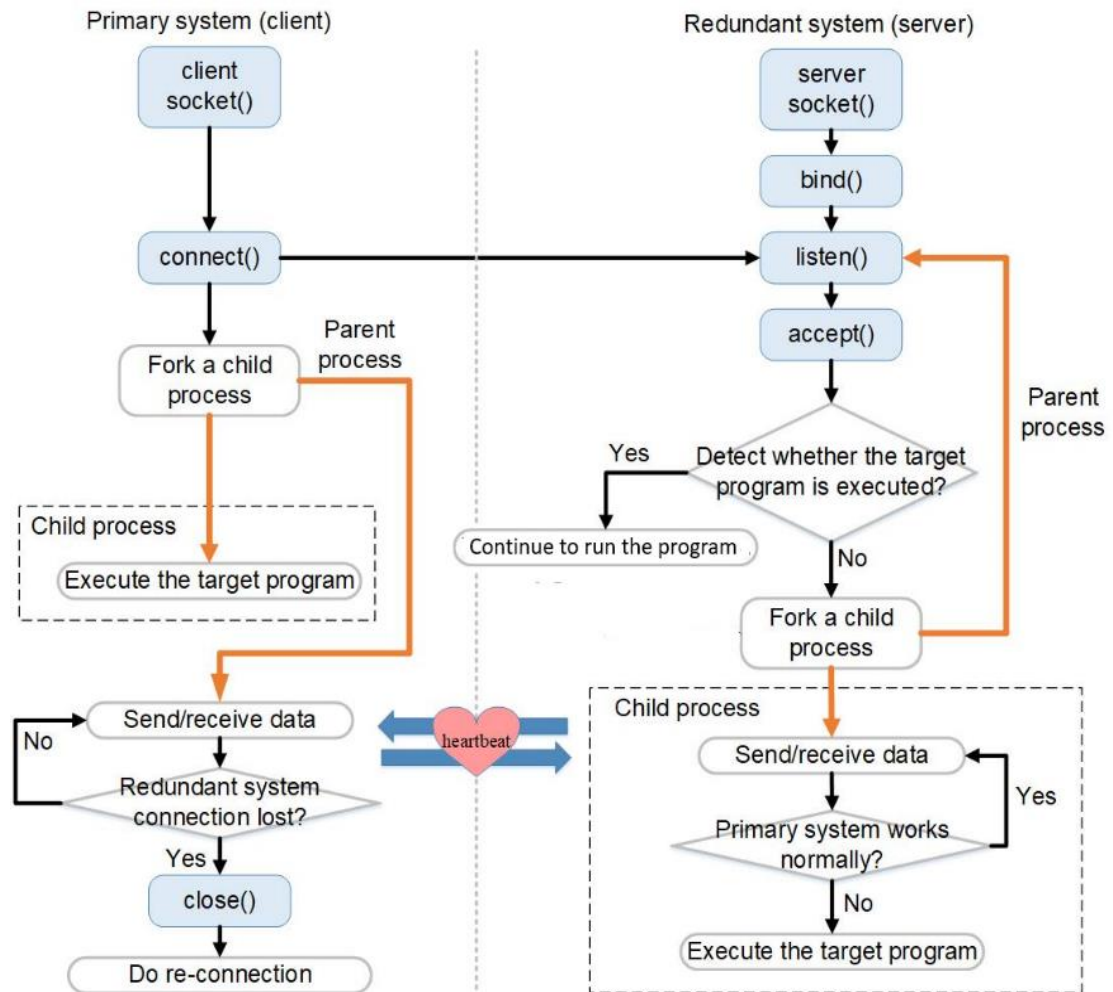


Figure 3.4: Execution flow between the primary and redundant systems.

3.1.1. Primary system:

The parent process of the primary system is responsible for handling the sending and receiving of heartbeat packets and detecting whether the redundant connection is disconnected. It will also fork a child process to execute the target program and check the connection through the heartbeat mechanism. If the redundant system connection is lost, the primary system will close the socket connection, and periodically try to connect to the redundant system again. The left part of Figure 3.4 illustrates the connection and execution process of the primary system.

Regarding the fault detection method, at first the redundant system acts as a monitor, sending heartbeat packets to the primary system regularly and the number of failures of the primary system. The purpose of sending heartbeat packets is to check whether the primary system is operating normally. When the primary system receives a heartbeat packet, it will respond to the packet to the redundant system. If the redundant system does not receive the response packet from the primary system within a period of time, it is considered that the primary system has failed and the recovery process is initiated. The purpose of transmitting the number of failures of the primary system is to let the primary system know whether it has a history of failures. This counter is very important because the primary system can know whether it wants to be a monitor through this number of failures. For example, at first the redundant system acts as a monitor, but after the primary system fails, the redundant system takes over the continued operation of the target program at this time. When the primary system restarts, it needs to know whether it wants to take over the target program or act as a monitor. If there is no mechanism to inform the primary system of the number of failures, the primary system will repeatedly execute the error situation, so the redundant system will record the number of failures of the primary system and send it to the primary system, so that the primary system can correctly switch to execute the target program or it is the executive monitor.

When the failure is restored, the watchdog function is responsible for the automatic restart of the Raspberry Pi after a system failure. To enable the watchdog function in the Raspberry Pi requires two parts. The first part is to edit the system configuration file `/boot/config.txt` and add `dtoverlay=watchdog=on` to the file in order to tell the Raspberry Pi to enable the watchdog function. In the second part, edit `/etc/modules` and add `bcm2708_wdog` to the file. After the Raspberry Pi is restarted, the hardware will automatically load the watchdog module. After successfully installing the watchdog, you also need to edit the `/etc/watchdog.conf` file, remove the watchdog-device annotation, and edit the content of the configuration file according to our needs. At last, we need to set the program to automatically start the target program function after restarting the Raspberry Pi. The Raspberry Pi has a file `/home/pi/.config/lxsession/LXDE-pi/autostart`, the Raspberry Pi will load the contents of the file and execute it after restarting. Therefore, we need to edit the program data and related parameters that are automatically started into this file. We will add the following two lines `@sleep(10)` and `@lxterminal -e`

/home/pi/build-Redundant_system_UI_v2-Desktop-Release/primary_system 2 192.168.50.24 /home/pi/test 5" (as in Figure 3.5). With the above parameter settings, the Raspberry Pi will automatically execute the primary system and redundant system programs after restarting. "@sleep" means waiting for ten seconds after rebooting. This waiting time is for the Raspberry Pi to connect to the network and then start the program. Otherwise, if the Raspberry Pi is not connected to another system, the socket connection will fail. There are a total of four different parameters in the last line, the first parameter is the mode selection, there are modes 1 and 2 to choose from, There are a total of four different parameters in the last line, the first parameter is the mode selection, there are modes 1 and 2 to choose from, the second parameter 192.168.50.24 is the ip-address of the redundant system, and the third parameter /home/pi /test is the execution path of the target program. The fourth parameter represents a checkpoint executed in a few seconds. Fill in 5 here to execute a checkpoint every 5 seconds. By adding them to the watchdog configuration file in order, the primary system and redundant system programs can be automatically started after the restart, and automatically connected to the monitor.

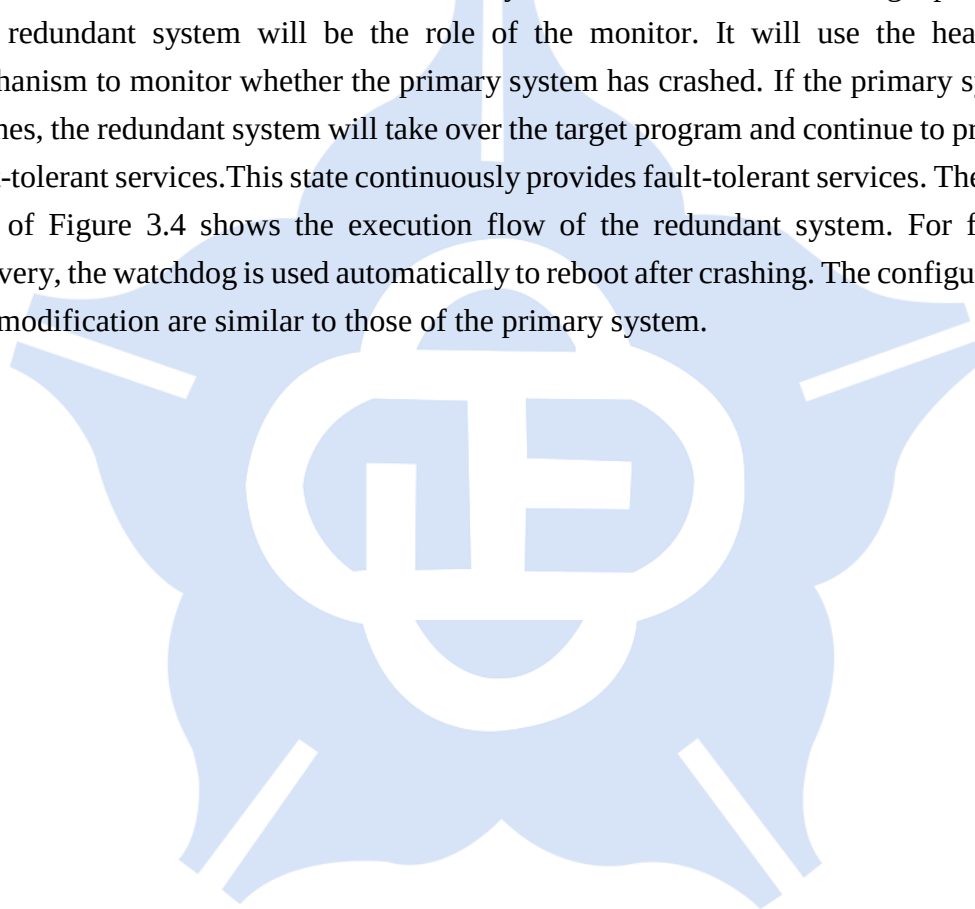
```
@lxpanel -profile LXDE-pi
@pcmanfm -desktop -profile LXDE-pi
@xscreensaver -no-splash
point-rpi
@sleep(10)
@lxterminal -e /home/pi/build-Redundant_system_UI_v2-Desktop-Release/primary_system 2 192.168.50.24 /home/pi/test 5
```

Figure 3.5:Contents of the auto-start configuration file.

3.1.2. Redundant system:

The redundant system has two operating states. In the first state, when the redundant system has taken over the task executed by the target program, the redundant system will continue to execute the target program, and the redundant system will also fork a child process for heartbeat checking. Since the redundant system is responsible for executing the target program, the primary system will be converted into a monitor.

The second state is that the redundant system does not execute the target program. The redundant system will be the role of the monitor. It will use the heartbeat mechanism to monitor whether the primary system has crashed. If the primary system crashes, the redundant system will take over the target program and continue to provide fault-tolerant services. This state continuously provides fault-tolerant services. The right side of Figure 3.4 shows the execution flow of the redundant system. For failure recovery, the watchdog is used automatically to reboot after crashing. The configuration and modification are similar to those of the primary system.



3.1.3 Role exchange:

We have already explained the individual work of the primary system and the redundant system, and then we can explain the overall role exchange operation process.

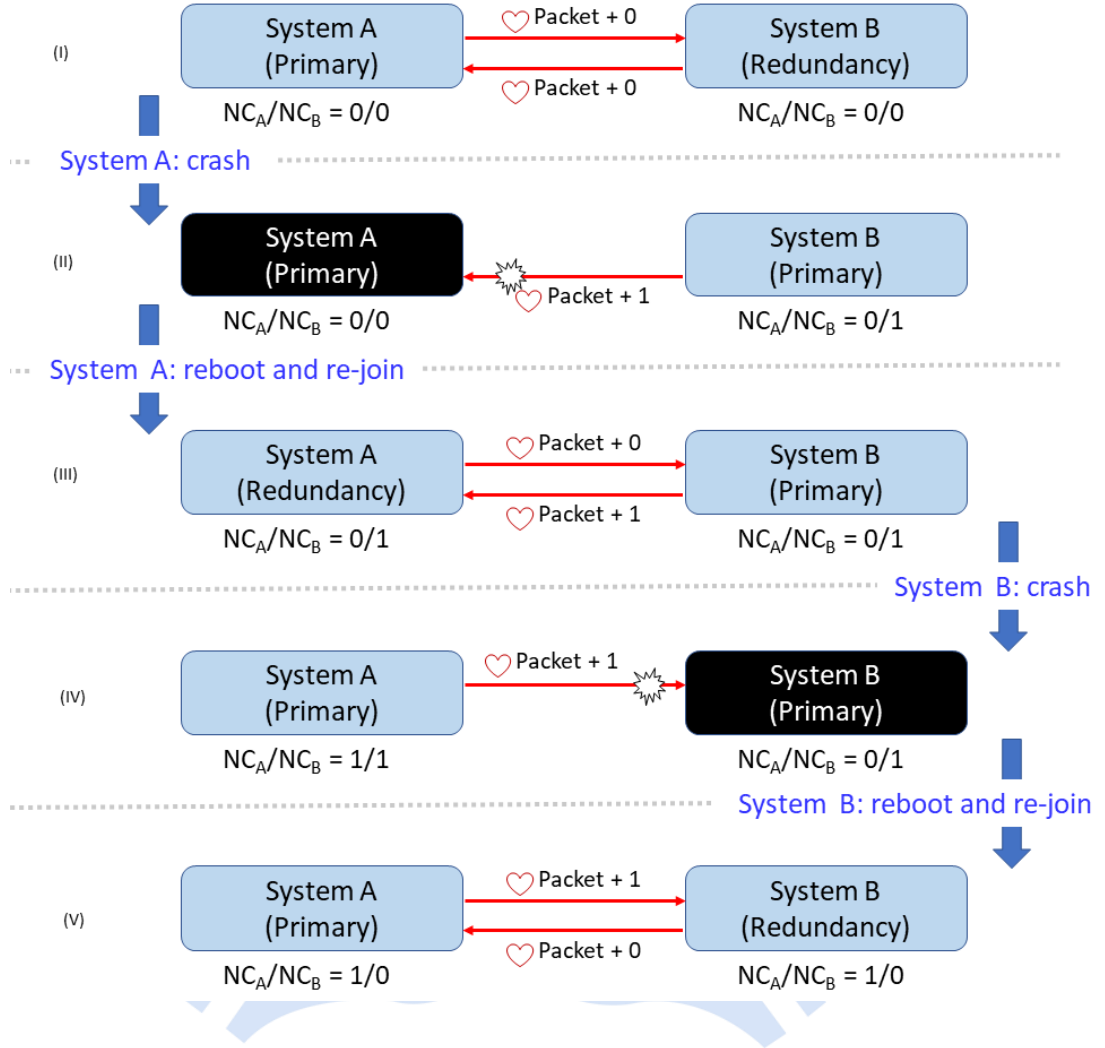


Figure 3.6: Example of role exchange.

In order to realize the role exchange, a system needs to know its status. It is difficult for a system itself to identify that it is rebooting for the first time or rebooting after a crash, whereas the other system can easily observe this issue. Accordingly, for each system, two NC (number of crashes) variables are used to identify its status and role: one is from self-observation, and the other is obtained from the other system.

Variable $NC_X(Y)$ represents the number of the crash of system Y observed by system X. The example in Figure 3.6 depicts the algorithm. For system A, $NC_A(A)$ is the number of the crash of system A, which is observed by system A itself. $NC_A(B)$ is the number of the crash of system B, which is observed by system A. System B also has two variables: $NC_B(A)$ and $NC_B(B)$. After a system reboots, its NC value observed by itself [i.e., $NC_A(A)$ of system A and $NC_B(B)$ of system B] is reset to 0. When reconnecting after the crash, the other system will send the correct NC value. This design means that a system can use the two NC values to identify its status and role.

Figure 3.6 (I) shows that systems A and B reconnect after their first reboot, and they have an NC value of 0. For system A, its NC value observed by the other system, $NC_B(A)$, is continuously transmitted from system B. System A continuously sends $NC_A(B)$ to system B. If system A is down, system B can detect that, and $NC_B(A)$ is incremented by 1 [as shown in Figure 3.6 (II)]. After system A has rebooted and rejoined, it resets $NC_A(A)$ to 0 and gets $NC_B(A)$ from system B. $NC_A(A)$ is 0, which represents that system A has just rebooted. $NC_B(A)$ represents the correct number of the crash of system A. According to the two values, system A knows that it has just been restarted from a crash and system B takes the role of the primary system. System A changes its role to the redundant system [as shown in Figure 3.6 (III)]. Figure 3.6 (IV) and (V) depict the situation of system B having crashed.

3.2. Checkpointing

In order to record the image (snapshot) of the running program, the image of the program can help us quickly restore the current state of the program. We need to use the checkpoint tool. Here we use DMTCP to create an image of the real-time program, and the user can set the program. If the system crashes, you can use the created program image to restore the program state.

```
$ dmtcp_launch -i 5 \  
  --with-plugin ./libdmtcp_mv_ckpt_to_gluster.so \  
  --ckptdir /dmtcp \  
  target_program
```

Figure 3.7: Command-line options used in DMTCP.

Figure 3.7 illustrates the command setting options used by DMTCP, option "-i 5" means that DMTCP will create an image of the program every 5 seconds, "-with-plugin" means to use a custom plug-in "libdmtcp_mv_ckpt_to_gluster.so", when we use DMTCP completes the image of the program, but it cannot be stored directly in the GlusterFS distributed file system, so we need to use the plug-in method to change the storage location. First, we write a .so file "libdmtcp_mv_ckpt_to_gluster.so", which can store the image of the program first In the local, copy a copy to GlusterFS. "-Ckptdir" is used to set the directory path to store the image. Finally, "target_program" is the path where the target program is executed.

```
$ /dmtcp/dmtcp_restart_script.sh \  
  --restartdir /dmtcp/ --no-strict-checking \  
  -h localhost
```

Figure 3.8: Command-line options for the DMTCP resume program.

Figure 3.8 illustrates the command line options using the DMTCP recovery program. When Figure 3.8 is executed, it will automatically create an image of the program. It will also create a shell script "dmtcp_restart_script.sh". This script records the restart command and the original program parameters. Therefore, through this script, we can quickly and conveniently restore the program. To restore the program in the primary system or redundant system, you need to change the host to the currently used system, so we set the host option through -h localhost, "-restartdir" is to read checkpoint images from the specified directory. "-no-strict-checking" means to prohibit uid checking of the checkpoint image. This setting allows different hosts to use the same image to do the recovery program.

Finally, we will recommend the user's checkpoint time settings. If the user's target program is a program that requires high real-time performance, we will recommend that the checkpoint time should be set to a smaller interval, because the intensive establishment of system snapshots can reduce the impact of recovery after a failure. On the contrary, if the user program only needs to restore the program state and continue to run, and does not need a high real-time program, you can choose to set the checkpoint time to a larger interval.

3.3. Dependable storage

GlusterFS provides open-source code and good horizontal scalability. GlusterFS provides users with three different types of volumes: 1. Distributed Glusterfs Volume, 2. Replicated Glusterfs Volume, 3. Striped Glusterfs Volume. We can use Replicated Glusterf Volume to store the same data in Different bricks, therefore, Replicated Glusterf Volume does not have to worry about damage to one of the bricks and cannot access the data in it, which can increase the reliability of data sharing.

```
1 $ sudo gluster peer probe $Redundant_IP
2 $ sudo gluster volume create gv0 replica 2 $Primary_IP:/srv/gv0 \
  $Redundant_IP:/srv/gv0 force
3 $ sudo gluster volume start gv0
4 $ sudo gluster volume set gv0 network.ping-timeout 1
5 $ sudo mkdir -p /mnt/gluster1
6 $ sudo mount -t glusterfs $Redundant_IP:/gv0 /mnt/gluster1
```

Figure 3.9: Command-line options used to configure GlusterFS .

Figure 3.9 illustrates the command line options used by GlusterFS. The first line adds the IP of the redundant system to the shared cluster. The second line is to create a volume named gv0, replica 2 represents the volume in the brick of the primary system and the brick of the redundant system using the Replicated Glusterfs Volume mode. This command is to automatically copy the same data in two different brick. The third line is to complete the creation of the volume, and then start it. The fourth line is to set the storage time of the volume on the network, as shown in Figure 3.10. Especially for the four lines of red text, we modify the value of "Ping-timeout to" to 1, which represents the waiting time before recovery, in order to reduce the network disconnection time between the client and the server.

If we use the default value of "Ping-timeout to" is 42 seconds, it will cause the time delay of the mobile system's image to GlusterFS to increase, so we set it to 1 to reduce the ping time and achieve rapid recovery. Line 5 is to create a directory dedicated to the gv0 volume. The sixth line mounts the directory named "/mnt/gluster1" to the gv0 volume. Whether it is the primary system or the redundant system, put the file in "/mnt/gluster1", and the file will be automatically copied to the bricks on both sides. Finally, edit the file "/etc/fstab" and add a line "192.168.50.24:/gv0 /mnt/gluster1 glusterfs defaults 0 0" to enable automatic mounting.

```
volume management
  type mgmt/glusterd
  option working-directory /var/lib/glusterd
  option transport-type socket,rdma
  option transport.socket.keepalive-time 10
  option transport.socket.keepalive-interval 2
  option transport.socket.read-fail-log off
  option ping-timeout 0
  option event-threads 1
#  option transport.address-family inet6
#  option base-port 49152
end-volume

volume gv0
  type protocol/client
  option ping-timeout 1
end-volume
```

Figure 3.10: Content of the glusterd.vol config file.

3.4. Software Rejuvenation Agent (SRA)

In the previous section, we have completed the fault-tolerant service consisting of the primary system and the redundant system. The system-level redundancy can ensure that the user program can continue to run when the user has hardware or man-made failure, and the execution performance will not be affected by the system crash. When the system has a slow and continuous failure of software aging, in the system-level redundancy, only the failure can be found, but it cannot be known in advance that the user program has signs of software aging. If the system crashes due to the aging of the software, it will take extra time and cost to exchange the roles of the operating system. Therefore, real-time monitoring of memory and real-time restart of user programs are important tasks. In [19], it can be seen that the IBM server has software aging due to long-term operation, leading to server failures and performance degradation. They use regular software rejuvenation tools to restart the system, and the server's run time has been significantly improved. We divide the explanation into two parts. The first part is about using the stress-ng tool to measure the user's memory limit space. The measured memory limit average value can help users set the threshold for rejuvenation. The second part explains how to add real-time memory monitoring to the primary system and redundant system.

3.4.1 Memory pressure process

First use the stress-ng pressure command. For example, "stress-ng --vm 1 --vm-bytes 1M", "--Vm 1" means to generate a worker cycle to call `malloc()/free()` to generate memory pressure, "--Vm-bytes 1M" means that the size of the pressurized memory is 1Mbytes this time. The operation flow of measuring the memory limit value is shown in Figure 3.11.

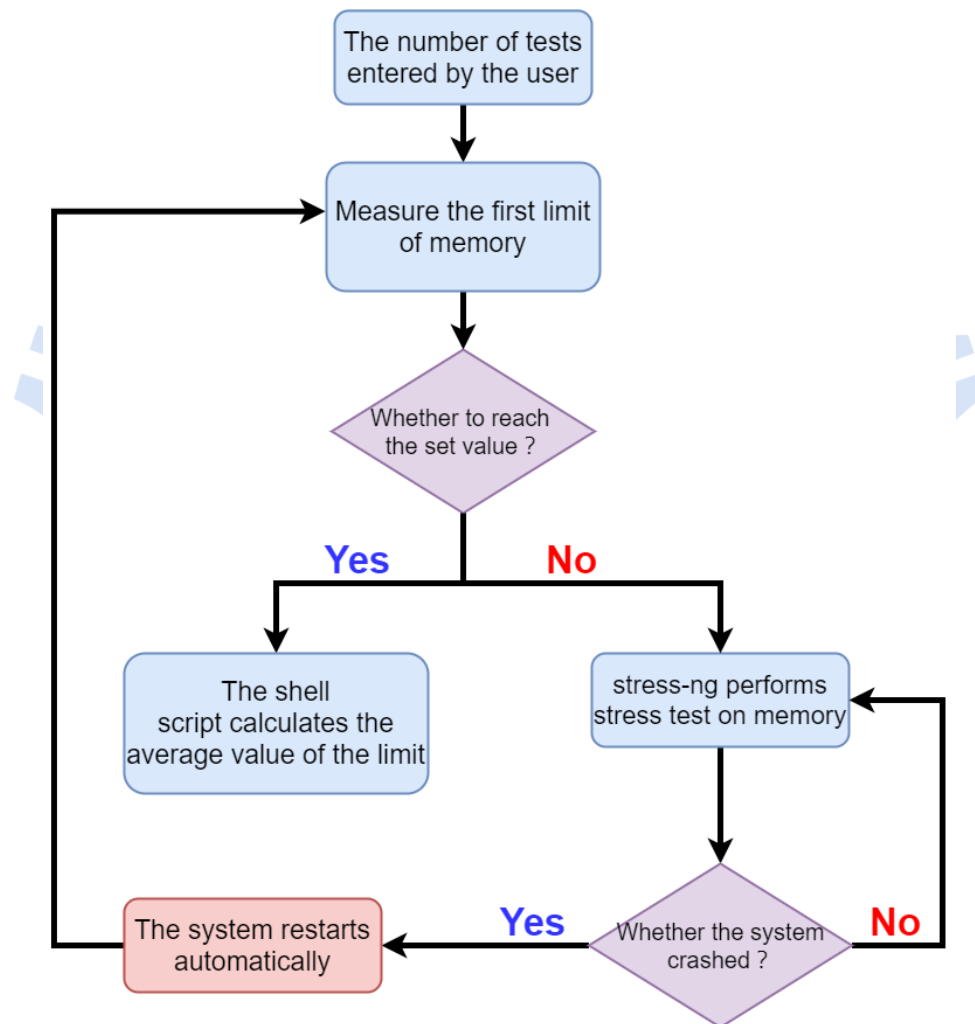
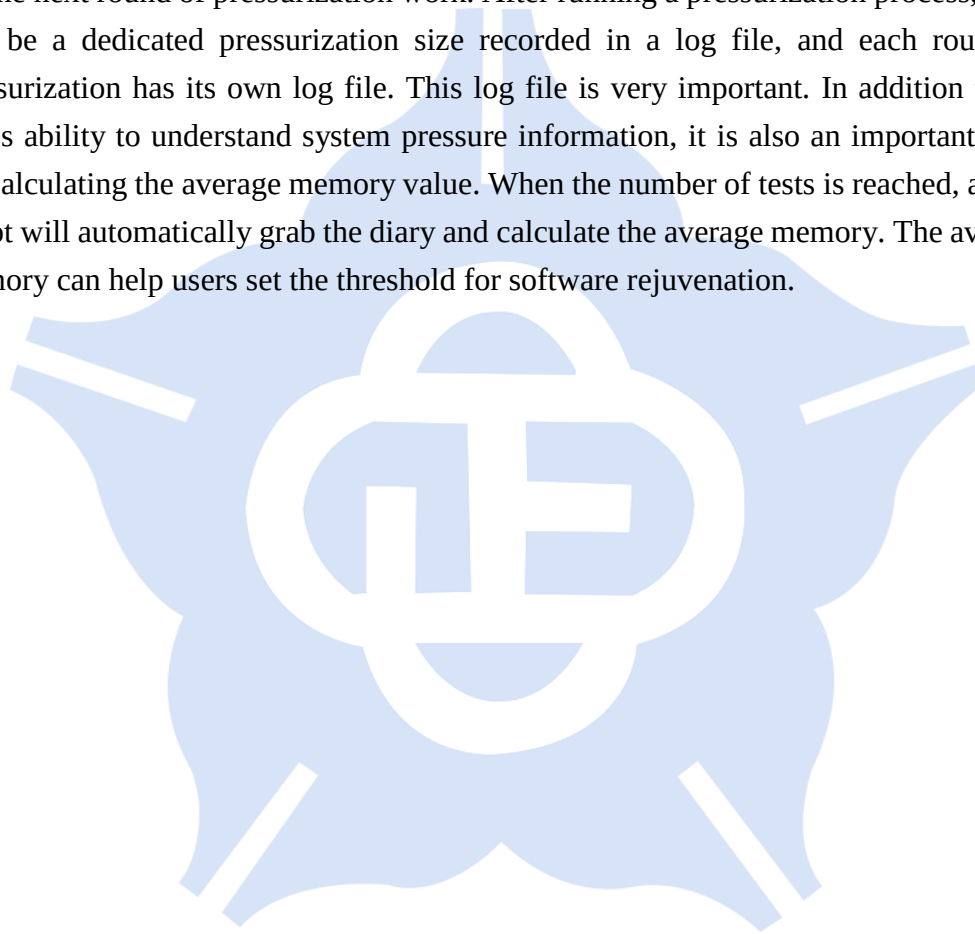


Figure 3.11: Flow chart of memory limit measurement.

When the user enters the desired number of measurements, it will first determine whether the number of measurements has been reached. If the number of tests has not been reached, the first limit of memory will be measured. We use `malloc` to apply for

an ever-increasing memory until malloc returns NULL, which means that the memory of the system has reached the allocation limit and we will get a memory limit value. However, there is a gap between this method and the actual memory. Therefore, stressing will use this first limit value as a pressure basis. The pressurization process uses the first limit value multiplied by the percentage. For example, the initial pressurization will be the first limit value multiplied by 1%, and the percentage value is increased in sequence until the system reaches a crash. The pressurization program will be restarted for the next round of pressurization work. After running a pressurization process, there will be a dedicated pressurization size recorded in a log file, and each round of pressurization has its own log file. This log file is very important. In addition to the user's ability to understand system pressure information, it is also an important basis for calculating the average memory value. When the number of tests is reached, a shell script will automatically grab the diary and calculate the average memory. The average memory can help users set the threshold for software rejuvenation.



3.4.2 Real-time memory monitoring

After the user finishes setting the threshold of software rejuvenation, in order not to increase the overhead of the system, therefore, only the primary system and redundant system are executing the target program, and the Software Rejuvenation Agent (SRA) will be activated. The execution flow of SRA is shown in Figure 3.12.

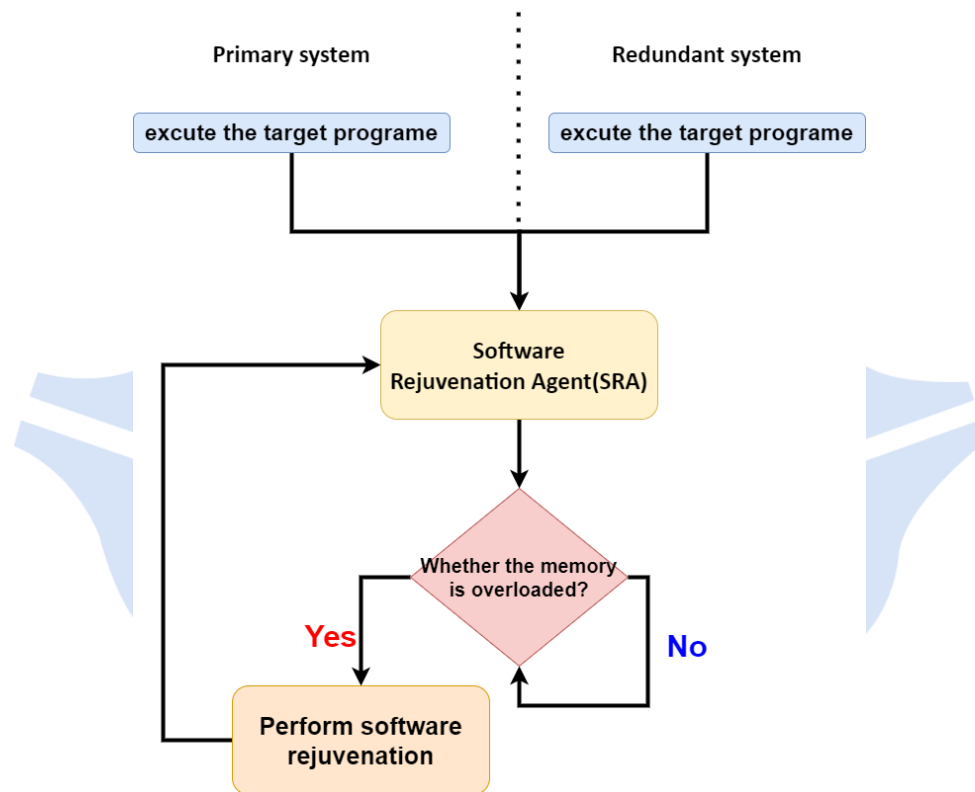


Figure 3.12: Software rejuvenation execution flow.

As shown in Figure 3.12, firstly, it will automatically grab the user's threshold, and use the top command to grab the memory size of the current user's program memory. Compare the two values. If it runs normally, it will display "ok" to remind the user. If it exceeds the user's threshold, "danger" will be displayed, and the user program will be restarted locally to clear the possible memory aging and keep the program running. Regarding the time interval for monitoring the user's memory usage, it will be used in accordance with the user's checkpoint time. For example, if the user sets a system snapshot to be created every 5 seconds, then in the 5th second, in addition to the system snapshot, a memory usage monitoring will also be run.

3.5. Assistant tool

For users without relevant knowledge background, it may be difficult to establish an environment step by step in accordance with the methods introduced earlier. Therefore, we have developed two user interfaces so that users can complete the construction of the entire system by simply inputting parameters without needing to understand the relevant details. The user only needs to prepare two pieces of Raspberry Pi hardware and burn the Raspberry Pi operating system into the SD card. Next, after starting the GUI executable file, the user can enter the required information and click a button to set it, and then it is easy to start the measurement of the memory limit and the system-level redundancy. The Assistant tool is divided into two parts. The first part is the combination of the GUI and the memory limit measurement, and the second part is the combination of the GUI and the system-level redundancy.

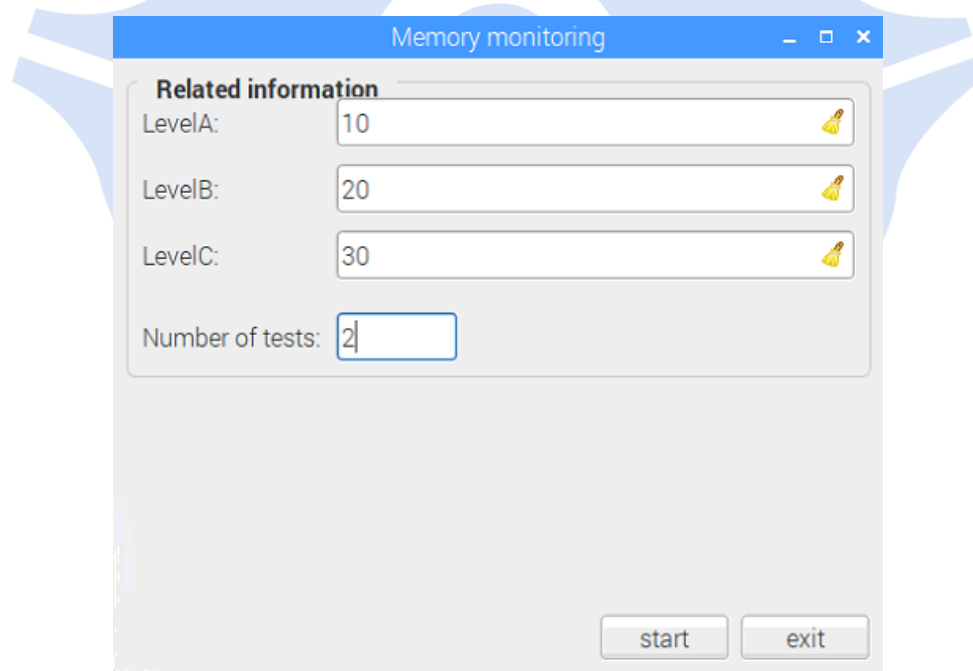


Figure 3.13: Screenshot of the memory measurement GUI.

As shown in Figure 3.13, First, the user can set the threshold of software rejuvenation through the level column. When the current user program memory usage is between A and B levels, the Raspberry Pi will display the word "ok" to remind the user. If the memory usage is greater than or equal to level B and less than level C, the

word "normal" will be displayed on the Raspberry Pi. Finally, if the memory usage is greater than or equal to level C, software rejuvenation will be executed, and the restart of the target program will be executed. The Test counts column represents the number of tests the user wants. For example, you can enter 2, press start to start the measurement, the entire measurement process will be executed twice. Moreover as shown in Figure 3.11, the average memory value and the minimum memory limit value are automatically calculated. Users can choose to view these two values to set the level column, or directly set the level column without obtaining the reference value through this measurement process.

Next, I will introduce the second GUI, which has the functions of installing software tools, setting up the environment configuration required to use the software tools, and executing the redundant programs we developed. Figure 3.14 shows the screenshot of the assistant tool. Next, we introduce the operation process. First, click the installation menu on the two Raspberry Pis to install GlusterFS, DMTCP, and watchdog. The order of installation does not affect the results. Secondly, the user chooses which Raspberry Pi is the primary system, and clicks the host field under the relevant information to select the primary system. The other one needs to be selected as a redundant system. Since WIFI dynamically allocates IP through DHCP, it may get a different IP for a long time or reconnect. The change of the IP address will cause a connection error in the GlusterFS file system. We need to set a static IP. Click the static IP option shown in Figure 3.15, fill in the field data in turn, and click the OK button to automatically set the static IP. After completing the static IP settings, you can set up GlusterFS.

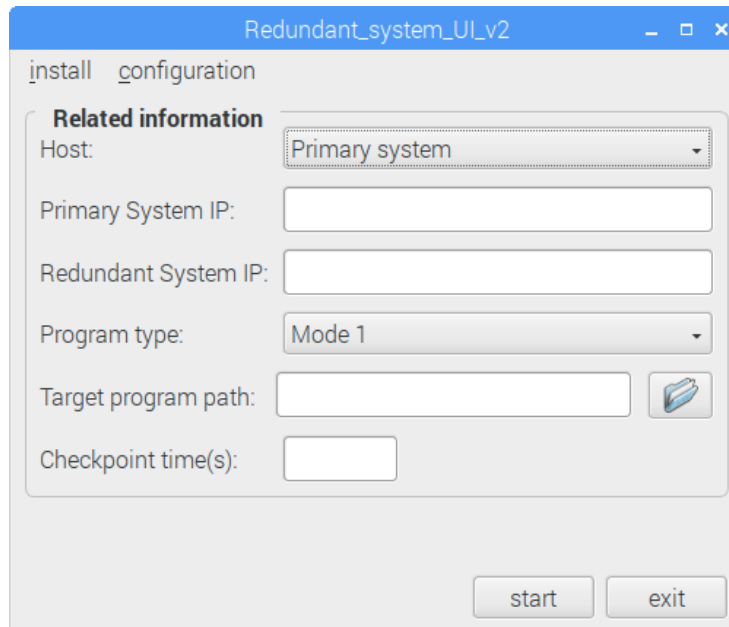


Figure 3.14: Screenshot of the fault-tolerant service GUI.

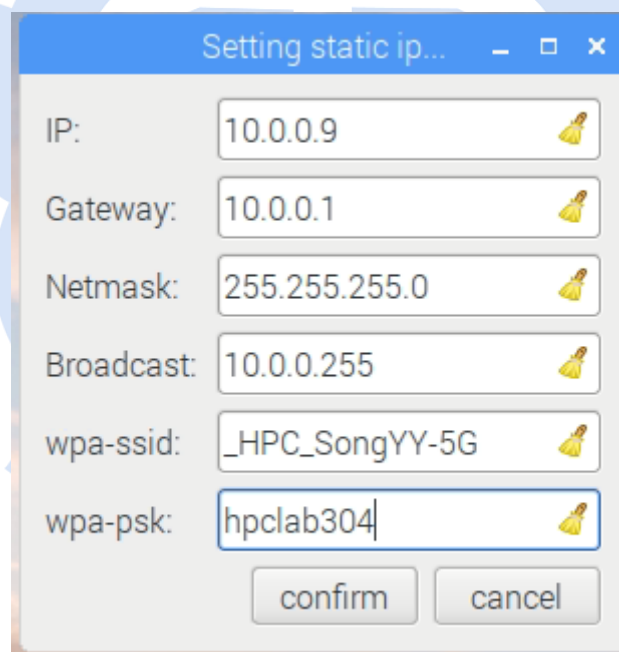


Figure 3.15: Screenshot of setting static IP.

Finally, select the gluster option in the configuration menu to configure the GlusterFS environment. It should be noted here that the primary system is set up first, and then the redundant system is set up. Figure 3.16 illustrates an example, enter the IP of the primary system and the redundant system. The target program execution type can choose mode-1 or mode-2. "/Home/pi/test" is the target program path. Checkpoint time is how often the user sets to perform checkpoints.

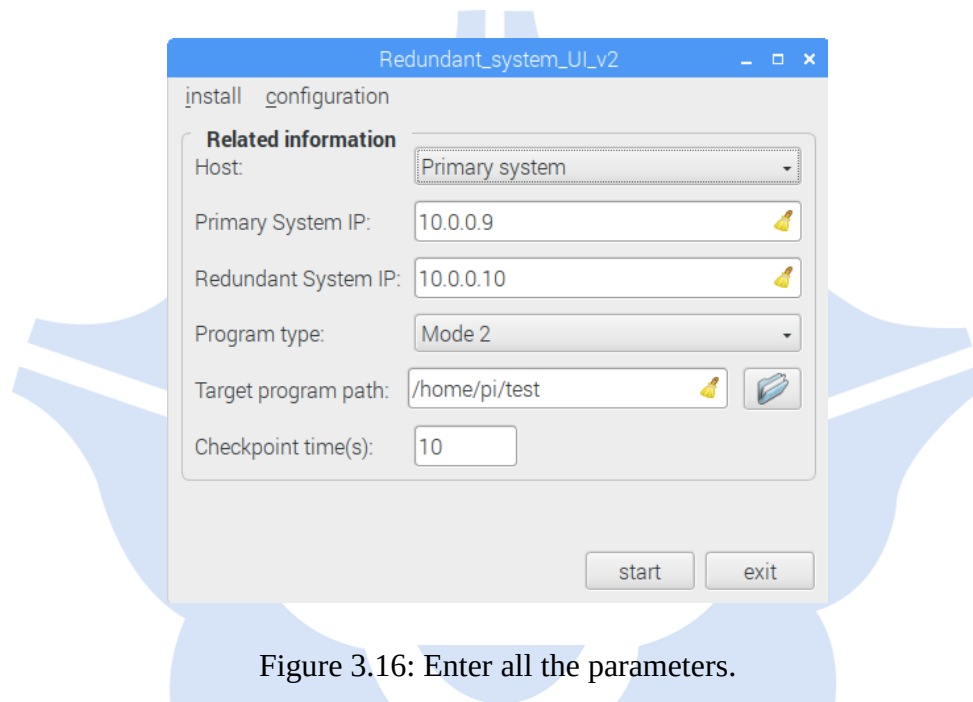


Figure 3.16: Enter all the parameters.

The redundant system needs to fill in information similar to the primary system. If it is the execution mode of mode-1, the target program needs to be placed on the redundant system. When the primary system crashes, the redundant system can execute the target program. The input of mode-2 program does not need to target program path and checkpoint time. Because when the primary system crashes, the redundant system uses the DMTCP checkpoint image to restore the program state. Furthermore, in this mode, both the primary system and the redundant system will use the threshold of the first GUI user as a parameter to execute SRA for memory protection and software rejuvenation. The last step is to press the start button to start using the redundant system and software rejuvenation tool we developed.

3.6. mtrace hook operation

First, we need to confirm the glibc version of the system, and compile and install it through Figure 3.17. We have written the entire installation process into a script. Users only need to modify the version of glibc, and the next installation can be done.

```
#!/usr/bin/bash
sudo apt-get install gcc -y
sudo apt-get install g++ -y
wget https://ftp.gnu.org/gnu/glibc/glibc-2.31.tar.gz
mkdir /home/pi/glibc-bulid/
tar -xvf /home/pi/glibc-2.31.tar.gz
cd /home/pi/glibc-bulid/
sudo apt-get install make -y
sudo apt-get install gawk -y
sudo apt-get install bison -y
../glibc-2.31/configure --prefix=/home/pi/glibc-bulid/
make -j3
sudo make install
```

Figure 3.17: Introduction to glibc installation.

Next, users have two choices. The first is to link the user program to our glibc library when compiling the program, for example, "gcc -g test.c -o test -Wl,--rpath=/home/pi/glibc-bulid/lib/ -Wl,--dynamic-linker=/home/pi/glibc-bulid/lib/ld-linux-x86-64.so.2". This command represents the use of dynamic linking of the library of the program named test to specify our glibc library. The second way is to replace the glibc library in the user's system with our glibc library, for example, "sudo ln -snf /home/pi/glibc-bulid/lib/libc-2.31.so /usr/lib/x86_64-linux-gnu/libc.so.6", this instruction represents replacing the glibc of the current system with our glibc library. You can run the automatic tracking function of mtrace using these two methods.

When the user calls `mtrace()` and `muntrace()`, it will enter `mtrace.c`. Figure 3.18 illustrates how `mtrace` uses hooks to intercept the information. First, it will determine whether the `mallfile` exists. `mallfile` represents the file name path of the log file output specified by the user. If it exists, proceed to the step of opening the file. And `save_default_hooks()` records the original hook pointers(`hook.c:151`). There is also use `set_trace_hooks()`(`hook.c:140`) to point the current hook to the function in `mtrace` for data collection.

```
void
mtrace (void)
{
    .....
    if (mallfile != NULL || mallwatch != NULL)
    {
        char *mtb = malloc (TRACE_BUFFER_SIZE);
        if (mtb == NULL)
            return;

        mallstream = fopen (mallfile != NULL ? mallfile : "/dev/null", "wce");
        if (mallstream != NULL)
        {
            malloc_trace_buffer = mtb;
            setvbuf (mallstream, malloc_trace_buffer, _IOFBF, TRACE_BUFFER_SIZE);
            fprintf (mallstream, "= Start\n");
            save_default_hooks ();
            set_trace_hooks ();
        }
        .....
    }
}
```

Figure 3.18: `mtrace` program snippet(`hook.c:304`).

It can be known from `set_trace_hooks()` that after the user calls `mtrace()`, `malloc` will be referenced to `tr_mallochook`. As shown in Figure 3.19, `set_default_hooks()` (`hook.c:131`) is used to transfer hooks back to the original `malloc.c`, in order to help users to allocate memory space normally. After executing the user's allocated memory, use `set_trace_hooks()` to transfer the hook back to `mtrace.c`, and use `tr_where(caller, info)` to collect `malloc` and `free` user information. Finally, use `fprintf` to write the memory size and memory address to the log file. Pay attention to the problem of locking here. Because `malloc` data needs to be recorded, this memory needs to be locked. When all operations are over, `__libc_lock_unlock(lock)` is needed for unlocking. After unlocking, the memory is handed over to the user program for subsequent use.

```
static void *
tr_mallochook (size_t size, const void *caller)
{
    void *hdr;
    Dl_info mem;
    Dl_info *info = lock_and_info (caller, &mem);
    set_default_hooks ();
    if (tr_old_malloc_hook != NULL)
        hdr = (void *) (*tr_old_malloc_hook) (size, caller);
    else
        hdr = (void *) malloc (size);
    set_trace_hooks ();
    tr_where (caller, info);
    /* We could be printing a NULL here; that's OK. */
    fprintf (mallstream, "+ %p %#lx\n", hdr, (unsigned long int) size);
    __libc_lock_unlock (lock);

    if (hdr == mallwatch)
        tr_break ();

    return hdr;
}
```

Figure 3.19: Program snippet of `tr_mallochook(hook.c:186)`.

As shown in Figure 3.20, `tr_where` will use a `buf` to record the user data of the structure `Dl_info`, `info->dli_saddr` is the malloc address of the record user, `info->dli_sname` is the name of the program that records the user, and uses `fprintf` to write the detailed information into the log file.

```
static void
tr_where (const void *caller, Dl_info *info)
{
    .....
    if (info->dli_sname != NULL)
    {
        .....
        buf[0] = '(';
        __stpcpy (__fitoa (caller >= (const void *) info->dli_saddr
                           ? caller - (const void *) info->dli_saddr
                           : (const void *) info->dli_saddr - caller,
                           __stpcpy (__mempcpy (buf + 1, info->dli_sname, len),
                                       caller >= (void *) info->dli_saddr
                                       ? "+0x" : "-0x"),
                           16, 0),
                ")");
    }
    fprintf (mallstream, "@ %s%s%s[%p] ",
            info->dli_fname ? : "", info->dli_fname ? ":" : "",
            buf, caller);
    .....
}
```

Figure 3.20: Program snippet of `tr_where(hook.c:77)`.

Finally, we will explain `muntrace()`, as shown in Figure 3.21. The key point is to close the `mallstream` file (That is, the `mallfile` file is closed) and use `set_default_hooks()` to return the hook to `malloc.c` to end all hook operations.

```
void
muntrace (void)
{
    if (mallstream == NULL)
        return;
    FILE *f = mallstream;
    mallstream = NULL;
    set_default_hooks ();

    fprintf (f, "= End\n");
    fclose (f);
}
```

Figure 3.21: Program snippet of `muntrace(hook.c:353)`.

3.6.1 Our modified version

This section will explain in order. First, we will explain how to use the hook mechanism to help us automatically mount `mtrace()`, then explain how to filter specific users of `malloc()` and `free()`, and finally explain how to output log files in real time. We modified `hook.c` to change the control flow of the entire program. As shown in Figure 3.22, when the user calls `malloc()` for the first time, initialization will be performed in sequence on the leftmost side of Figure 3.22. When the user is not calling `malloc()` for the first time, he will not use the hook again to initialize `ptmalloc`. Therefore, we can use this feature, as shown on the far right of Figure 3.22. The modified version is to use a hook to enable `mtrace()` tracking. As shown in Figure 3.23, automatic mounting can be achieved by adding `mtrace()` to `malloc_hook_ini`.

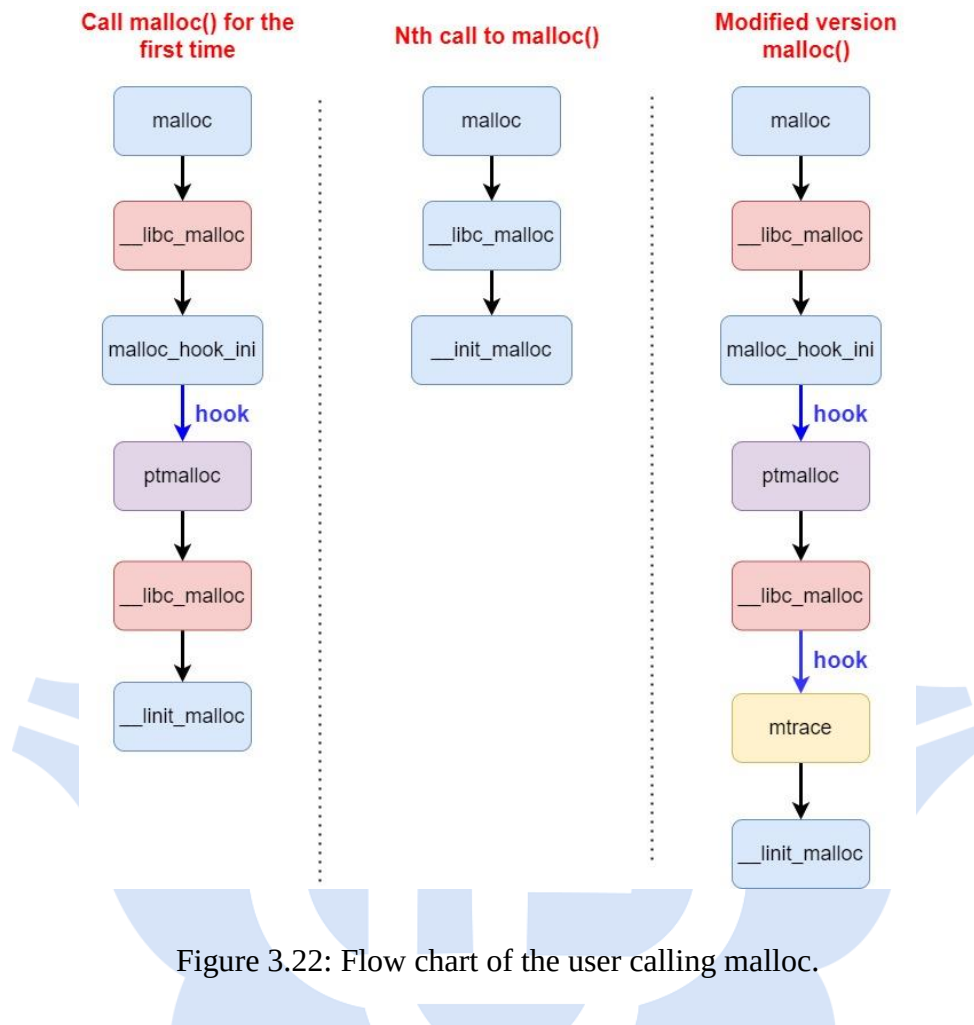


Figure 3.22: Flow chart of the user calling malloc.

```

#include <mcheck.h>

static void *
malloc_hook_ini (size_t sz, const void *caller)
{
    __malloc_hook = NULL;
    ptmalloc_init ();
    mtrace();
    return __libc_malloc (sz);
}

```

Figure 3.23: Program snippet of malloc_hook_ini(hook.c:28).

Then, we modified `tr_mallochook`, `tr_freehook`, and `tr_where` in `mtrace.c`, and added discriminants to help us filter unnecessary messages. When I first came into contact with `mtrace.c`, because there was no filtering of other noise, the amount of noise directly paralyzed the virtual machine, so this step is very important. As shown in Figure 3.24, you can specify the tracking target by adding it to each collection function. The red text is the name of the program you want to track. Finally, we use `fsync` to write to the disk to ensure that the modified content is written to the disk immediately.

```
int length = strlen(info->dli_fname) + 1;
/* We could be printing a NULL here; that's OK. */
char text1[length];
strcpy (text1, info->dli_fname);
if ((strcmp(text1, "/usr/local/lib/dmtp/libdmtp_alloc.so") == 0) || (strcmp(text1, "./test1") == 0)) {
    fprintf (mallstream, "+ %p %#lx %s\n", hdr, (unsigned long int) size, text1);
    fflush(mallstream);
    int wfd=0;
    wfd = fileno(mallstream);
    fsync(wfd);
}
```

Figure 3.24: Filter program fragments of user programs.

After the above modification, our memory tracking work is completed. When you choose the version developed by us, you can get the memory-related log files in real time.

Chapter 4 Experiment

This chapter will first introduce the experimental configuration and benchmark program, and then show the actual operation screenshots of the primary system and the redundant system. It also measures the time it takes to restart the target program when the primary system crashes, and the size of the checkpoint image created each time a checkpoint is executed. Moreover when the memory usage exceeds the threshold set by the user, the actual running screenshot of the software rejuvenation is started. Finally, we introduce the test results of GUN mtrace in other papers. The following subsections will introduce the details in order.

4.1. Environment configuration

Our implemented technology was evaluated on the Raspberry Pi 4 model platform. The specific configuration of the experimental environment is shown in Table 4-1.

Table 4-1: Raspberry Pi-related parameters.

Item	Description
SoC	Broadcom BCM2711
CPU	Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
RAM	2GB LPDDR4-2400 SDRAM
OS	Linux 4.14.73-v7+
Compiler	gcc 6.3.0
Glibc	glibc-2.25
DMTCP	Version: 3.0
GlusterFS	Version: 3.8.8 (node:2, brick: 2, volume: 2)

The benchmark suite tested has two temperature sensor programs, MiBench [33], DSPstone [34] and one hand-coding program, as shown in Table 4-2.

Table 4-2: Benchmark programs.

Benchmark	Program type	Description
simpletest.py	Mode 1	Adafruit_Python_MAX31855 temperature sensor
dht11_example.py	Mode 1	DHT11 temperature and humidity sensor
basicmath_small	Mode 2	Basic math (MiBench)
basicmath_large	Mode 2	Basic math (MiBench)
isqrt	Mode 2	MiBench-basic math (Function)
rad2deg	Mode 2	MiBench-basic math (Function)
matrix2	Mode 2	Generic matrix (DSPstone)
test	Mode 2	Count number (from 0) every second (hand coded)

4.2. Experimental results

The experimental results consist of four parts. The first part shows the mode-1, the sensor temperature measurement procedure, and mode-2, the program state needs to be restored. The second part shows the experimental data, and the third part shows a screenshot of the actual running of the software rejuvenation when the memory usage exceeds the threshold set by the user. The fourth part shows the experimental data results of mtrace in other papers.

4.2.1 The actual operation screenshots

Mode-1 evaluation

Figure 4.1 is a screenshot of simpletest.py of the Adafruit_Python_MAX31855 temperature sensor executed by the primary system. In order to test fault-tolerant services, we used three methods. One is to directly press ctrl-c to interrupt the target program. Another is to halt then reboot. The other is to open the terminal and use the fork bomb command to crash the primary system. In these three situations, the redundant system can detect the failure of the primary system and display the information, and then execute the temperature and humidity measurement program.

```
---initial!---  
  
---connection success---  
reconnection: 0  
1  
Press Ctrl-C to quit.  
count: 1  
Thermocouple Temperature: 25.000*C  
count: 2  
Thermocouple Temperature: 25.000*C  
count: 3  
Thermocouple Temperature: 25.000*C  
count: 4  
Thermocouple Temperature: 25.000*C  
count: 5  
Thermocouple Temperature: 25.000*C  
count: 6  
Thermocouple Temperature: 25.000*C  
count: 7  
Thermocouple Temperature: 25.000*C  
count: 8  
Thermocouple Temperature: 25.000*C  
count: 9  
Thermocouple Temperature: 25.000*C
```

Figure 4.1: Screenshot on the primary system.

Figure 4.2 In the case of normal operation of the primary system, the redundant system displays a message that the primary system remains active. If the primary system fails, the redundant system immediately prints an error message and executes the DHT11 temperature and humidity sensor program. Wait for the primary system after the watchdog restarts, and the primary system will change its role to monitor.

```
primary system keep alive 1
primary system keep alive 1
primary system keep alive 1
primary system keep alive 1
primary system is die!: Success
Last valid input: 2019-05-13 17:17:54.361991
Temperature:24 C,75 F
Humidity: 30 %
Last valid input: 2019-05-13 17:17:55.442173
Temperature:25 C,77 F
Humidity: 15 %
Last valid input: 2019-05-13 17:17:56.522526
Temperature:25 C,77 F
Humidity: 20 %
Last valid input: 2019-05-13 17:17:58.682000
Temperature:25 C,77 F
Humidity: 21 %
Last valid input: 2019-05-13 17:17:59.762504
Temperature:25 C,77 F
Humidity: 21 %
```

Figure 4.2:Screenshot on the redundant system.

Figure 4.3 shows the execution screenshot of the primary system. When a failure occurs when the redundant system is the monitor, the primary system will display that the redundant system has lost connection, and will reconnect to the redundant system periodically. When a failure occurs when the primary system is the monitor, the redundant system will also display the same message and wait for the primary system to make a connection request again.

```
count: 75
Thermocouple Temperature: 24.750°C
count: 76
Thermocouple Temperature: 24.750°C
count: 77
Thermocouple Temperature: 24.750°C
count: 78
Thermocouple Temperature: 24.750°C
---connection success---
reconnection: 0
count: 79
Thermocouple Temperature: 24.750°C
count: 80
Thermocouple Temperature: 24.750°C
count: 81
Thermocouple Temperature: 24.750°C
count: 82
Thermocouple Temperature: 24.750°C
count: 83
Thermocouple Temperature: 24.750°C
```

Figure 4.3:Screenshot on the primary system during the redundant system crash.

Mode-2 evaluation

This section is to test the type of mode-2 program. This program is a hand-coded program. It will start counting every second from 0 and print the numbers on the screen. Figure 4.4 shows the normal operation of this program on the primary system. Because the checkpoint time is set to 5 seconds, the primary system will create a system image file in the 4th second. This operating condition is initialization. Therefore, the primary system runs the target program, and the redundant system monitors the primary system.

```
device = socket:[390878]
[40000] WARNING at socketconnection.cpp:236 in TcpConnection; REASON='JWARNING((domain == AF_INET || domain == AF_UNIX || domain == AF_INET6) && (
type & 077) == SOCK_STREAM) failed'
domain = 0
type = 0
protocol = 0
[40000] NOTE at processinfo.cpp:229 in growStack; REASON='bottom-most page of stack (page with highest address) was
invisible in /proc/self/maps. It is made visible again now.'
0 1 2 3 4
*** The plugin mv_ckpt_to_gluster.c is being called before checkpointing. ***
*** The plugin mv_ckpt_to_gluster.c has now been checkpointed. ***
***move finished***
5 6 7
```

Figure 4.4:Primary system executes the target program.

Figure 4.5 shows a screenshot of the redundant system. In the figure, it is checking whether the primary system is normal. Next, use the same three evaluations (ctrl-c, shutdown, and fork bomb commands) as in mod-1 to evaluate the proposed method. If the redundant system detects a failure of the primary system, it will display a failure message of the primary system on the screen, as shown in Figure 4.6. Then use the checkpoint image to restore the target program. You can see from the way that the number continues to run from 5 seconds later, instead of starting from 0 seconds.

```
pi@raspberrypi-redundancy:~/heartbeat $ ./redundant_system_server_mode2
---initial!---
wait..
---connection success---
reconnection:0
---Child Process:1663
child process chdpid: 1663
this is a new thread,you got connected
fd=6
primary system keep alive 1
primary system keep alive 1
primary system keep alive 1
```

Figure 4.5:Redundant system operation monitor.

```

--
primary system keep alive 1
primary system keep alive 1
primary system keep alive 1
primary system is die!: Success
5 6 7 8 9
*** The plugin mv_ckpt_to_gluster.c is being called before checkpointing. ***
*** The plugin mv_ckpt_to_gluster.c has now been checkpointed. ***
***move finished***
10 11 12 13 14
*** The plugin mv_ckpt_to_gluster.c is being called before checkpointing. ***
*** The plugin mv_ckpt_to_gluster.c has now been checkpointed. ***
***move finished***
--

```

Figure 4.6: Redundant system detected primary system failure.

Figure 4.7 shows a screenshot of the redundant system. If the primary system restarts and reconnects to the redundant system, the redundant system will continue to execute the target program. Finally, Figure 4.8 shows a screenshot of the primary system. It will transform the role into a monitor, monitor whether the redundant system is normal, and display the message that the primary system is normally alive on the screen. If a failure occurs again, the role will be changed again to ensure the continuity of fault-tolerant services.

```

*** The plugin mv_ckpt_to_gluster.c is being called before checkpointing. ***
*** The plugin mv_ckpt_to_gluster.c has now been checkpointed. ***
***move finished***
45 46 47 48 49
*** The plugin mv_ckpt_to_gluster.c is being called before checkpointing. ***
*** The plugin mv_ckpt_to_gluster.c has now been checkpointed. ***
***move finished***
50 ---connection success---
reconnection:1
kill child 1666
CMD:kill 1663
kill dmtcp
[40000] ERROR at dmtcpmessage.cpp:56 in assertValid; REASON='JASSERT(strcmp(DMTCP_MAGIC_STRING, _magicBits) == 0) failed'
_magicBits =
Message: read invalid message, _magicBits mismatch. Did DMTCP coordinator die uncleanly?
test (40000): Terminating...
--Child Process:1766
child process chdpid: 1766
this is a new thread,you got connected
redundant system keep alive 1
redundant system keep alive 1
redundant system keep alive 1
redundant system keep alive 1
redundant system keep alive 1

```

Figure 4.7:Redundant system executes the target program.

```

I'm working, flag:1 ---initiall... ==-connection success---
reconnection: 1
againagainagainagainagainagainagain
redundant system alive
redundant system alive
redundant system alive
redundant system alive

```

Figure 4.8:Primary system becomes a monitor.

4.2.2 Screenshot of the actual operation of software rejuvenation

This section also uses the Mode-2 test program, adding memory leaks to the original test. Test our memory monitoring and software rejuvenation functions. Figure 4.9 shows that the primary system will monitor the user's memory usage in real time when it is running. If it is within the range specified by the user, the word ok will be displayed on the screen. When the gate threshold set by the user is exceeded, it will be shown in Figure 4.10 that the primary system starts software rejuvenation and restarts the program, so you can see that the program starts running from 0 seconds. Clear the software aging caused by possible memory leaks.

```
dmtcp/ Contains file: ckpt_test_12085f9b-40008-ffffffff5ae65fb.dmtcp
dmtcp_restart_script_12085f9b-40 800-ffffffffffec6a6b3e.sh dmtcp_restart_script.sh
1*** move finished*** 5 6 7 *** The plugin mv_ckpt_to_gluster.c is being called
before checkpointing. *** *** The plugin mv_ckpt_to_gluster.c has now been
checkpointed. *** dmtcp/ Contains file: ckpt_test_12085f9b-40000-
ffffffff5ae65fb.dmtcp dmtcp_restart_script_12085f9b-40 800-ffffffffffec6a6b3e.sh
dmcp_restart_script.sh Jok ***move finished *** 8 9 ok 10 *** The plugin
mv_ckpt_to_gluster.c is being called before checkpointing. *** *** The plugin
mv_ckpt_to_gluster.c has now been checkpointed. *** dmtcp/ Contains file:
Ickpt_test_12085f9b-40000-ffffffff5ae65fb.dmtcp cmtcp_restart script 12085f9b-
48 800-ffffffffffec6a6b3e, sh dmtcp_restart_script.sh
```

Figure 4.9:Software Rejuvenation running screenshot.

```
device = pipe: [48130] [48800] WARNING at
socketconnection.cpp:236 in TcpConnection; REASON='JWARNING((
domain == AF_INET || domain == AF_UNIX || domain == AF_INET6)
&& (type & 077) == SOCK_STREAM) failed'
domain = 8 type = 0
protocol = 0 [48800] NOTE at socketconnlist.cpp:220 in scanFor
PreExisting; REASON='found preexisting socket... will not be
restored
fd = 9
device = pipe:[48150] [48800] WARNING at
socketconnection.cpp:236 in TcpConnection; REASON='JWARNING((
domain == AF_INET || domain == AF_UNIX || domain == AF_INET6)
&& (type & 077) == SOCK_STREAM) failed'
domain = 8 type = @
protocol = 0
0 1 2 ok
3 4
*** The plugin mv_ckpt_to_gluster.c is being called before checkpointing. ***
```

Figure 4.10: Screenshot of software sejuvenation trigger.

If a system crash occurs in this operating mode, the role transition process will be the same as Mode-2, so that users can have more options for the operating mode.

4.2.3 Experiment data

This part is the result of experimental data, which evaluates the time taken by the target program from the primary system crash to the restart of the target program in the redundant system, and the size of the checkpoint image. We tested it ten times on six benchmarks, and the checkpoint time was 5 seconds and 10 seconds. Use the fork bomb command to test the time it takes for the primary system to crash until the redundant system takes over the target program. Because the target program is executed on both systems, the target program will not stop the system condition under normal circumstances. Therefore, our method is to open another terminal on the primary system and the redundant system, and use the date "+%S" command to get the number of seconds on the current system. The first system time is taken from the primary system crash, the second time is taken from the redundant system to restart the test program, and then the second time is subtracted from the first time to get the restart time. The recovery time of SRA is the time from system monitoring to memory overload to completion of restarting the program. For the test method, we used [35] to add memory leaks to the same benchmark, and use the stress-ng command introduced earlier to help us create an environment where memory leaks lead to software aging.

Table 4-3: Benchmark test (checkpoint time = 5), size of executable code =8256 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " () { : & } ; : "
2205262	5	7.6
2205266	5	10.8
2205268	5	8.8
2205255	5	9.6
2205271	6	8.1
2205267	6	8.8
2205268	6	9.3
2205274	5	8.3
2205255	6	7
2205267	6	8.5
Average: 2205265.30	5.5	8.7

Table 4-4: Benchmark test (checkpoint time = 10), size of executable code =8256 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " () { : & } ; : "
2206726	5	10
2206675	6	10.8
2206611	6	7.2
2206625	6	6.9
2206607	5	7.8
2206620	5	9.1
2206613	5	7.6
2206622	5	7.7
2206605	7	7.4
2206617	6	10.5
Average: 2206632.1	5.6	8.5

Table 4-5: Benchmark basicmath_large (checkpoint time = 5), size of executable code = 12656 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " () { : & } ; : "
2211761	5	7.8
2211760	6	9.4
2211766	6	8.5
2211761	6	7.7
2211762	5	7.1
2211760	5	7.1
2211761	5	9.4
2211759	6	9.3
2211762	6	9.6
2211761	6	8.3
Average: 2211761.3	5.6	8.4

Table 4-6: Benchmark basicmath_large (checkpoint time = 10), size of executable code = 12656 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " () { : & } ; : "
2212353	5	8.1
2212151	6	7.7
2212123	6	9.1
2212108	5	8.8
2212119	4	7.8
2212113	6	7.1
2212118	5	9.5
2212127	5	10.1
2212123	5	10.2
2212127	4	7.6
Average: 2212146.2	5.1	8.6

Table 4-7: Benchmark basicmath_small (checkpoint time = 5), size of executable code = 8560 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " (){:& }; : "
2211563	6	8.7
2211585	5	9.6
2211558	6	7.6
2211560	6	9.4
2211550	4	9.5
2211559	6	7.4
2211546	5	10.3
2211553	6	7.3
2211545	4	7.6
2211542	5	9.8
Average: 2211556.1	5.3	8.7

Table 4-8: Benchmark basicmath_small (checkpoint time = 10), size of executable code = 8560 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " (){:& }; : "
2211610	5	10.3
2211651	5	7.5
2211541	6	7.7
2211547	6	10.3
2211554	5	8.4
2211553	6	7.8
2211549	5	8.4
2211546	6	7.2
2211550	6	7.4
2211552	6	7.6
Average: 2211565.3	5.6	8.3

Table 4-9: Benchmark isqrt (checkpoint time = 5), size of executable code= 8208 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " (){ : :& }; : "
2210454	5	8.9
2210418	4	7.8
2210405	5	8.9
2210391	5	8.8
2210394	5	7
2210405	6	9.7
2210400	4	8.9
2210403	5	8.2
2210410	5	9.5
2210390	6	8.8
Average: 2210407	5	8.6

Table 4-10: Benchmark isqrt (checkpoint time = 10), size of executable code= 8208 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " (){ : :& }; : "
2211613	5	8
2211610	6	7.1
2211651	5	6.7
2211541	5	9.7
2211547	6	9.6
2211554	6	6.6
2211553	4	9.8
2211549	6	7
2211546	5	9.1
2211550	5	9.8
Average: 2211571.4	5.3	8.3

Table 4-11: Benchmark rad2deg (checkpoint time = 5), size of executable code = 8336 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " () { : & } ; : "
2210510	5	9.6
2210518	5	8.8
2210487	6	7.8
2210479	4	7.4
2210461	5	9.6
2210473	4	9.5
2210461	6	7.7
2210474	6	9.5
2210465	5	8.1
2210462	4	7.9
Average: 2210479	5	8.6

Table 4-12: Benchmark rad2deg (checkpoint time = 10), size of executable code = 8336 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " () { : & } ; : "
2210336	6	9.4
2210285	5	8.5
2210288	4	9.2
2210273	5	8.2
2210266	5	9.9
2210273	5	7.4
2210287	6	8.1
2210272	6	7.7
2210265	5	7.8
2210267	6	7.7
Average: 2210281.2	5.3	8.4

Table 4-13: Benchmark matrix2 (checkpoint time = 5), size of executable code = 8344 bytes.

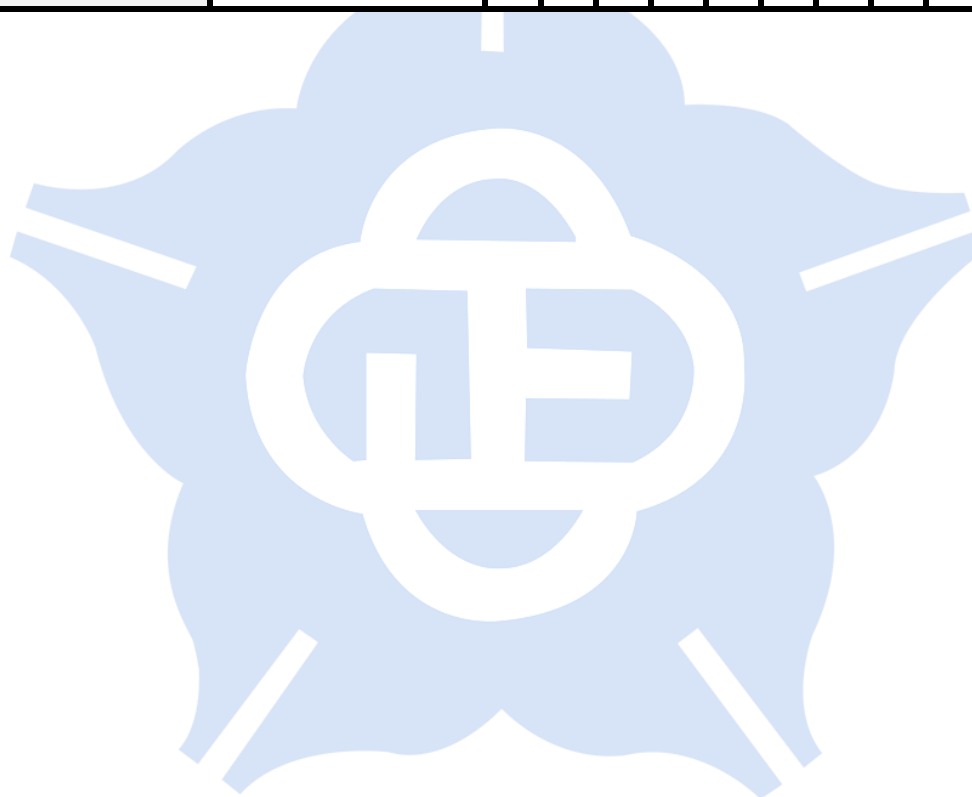
Checkpoint image file size	restart time (ctrl+c)	restart time " () { : & } ; : "
2210854	5	7.6
2210862	5	9.4
2210826	5	10.6
2210794	6	7
2210829	6	10.2
2210795	5	6.9
2210817	4	9.4
2210812	6	10.7
2210818	5	6.9
2210812	6	7.3
Average: 2210821.9	5.3	8.6

Table 4-14: Benchmark matrix2 (checkpoint time = 10), size of executable code = 8344 bytes.

Checkpoint image file size	restart time (ctrl+c)	restart time " () { : & } ; : "
2210579	6	9.6
2210597	6	10.3
2210508	4	6.9
2210489	5	7.1
2210479	5	9.5
2210491	5	9.2
2210495	4	9.3
2210494	5	10.8
2210484	6	7.3
2210475	5	8.8
Average: 2210509.1	5.1	8.9

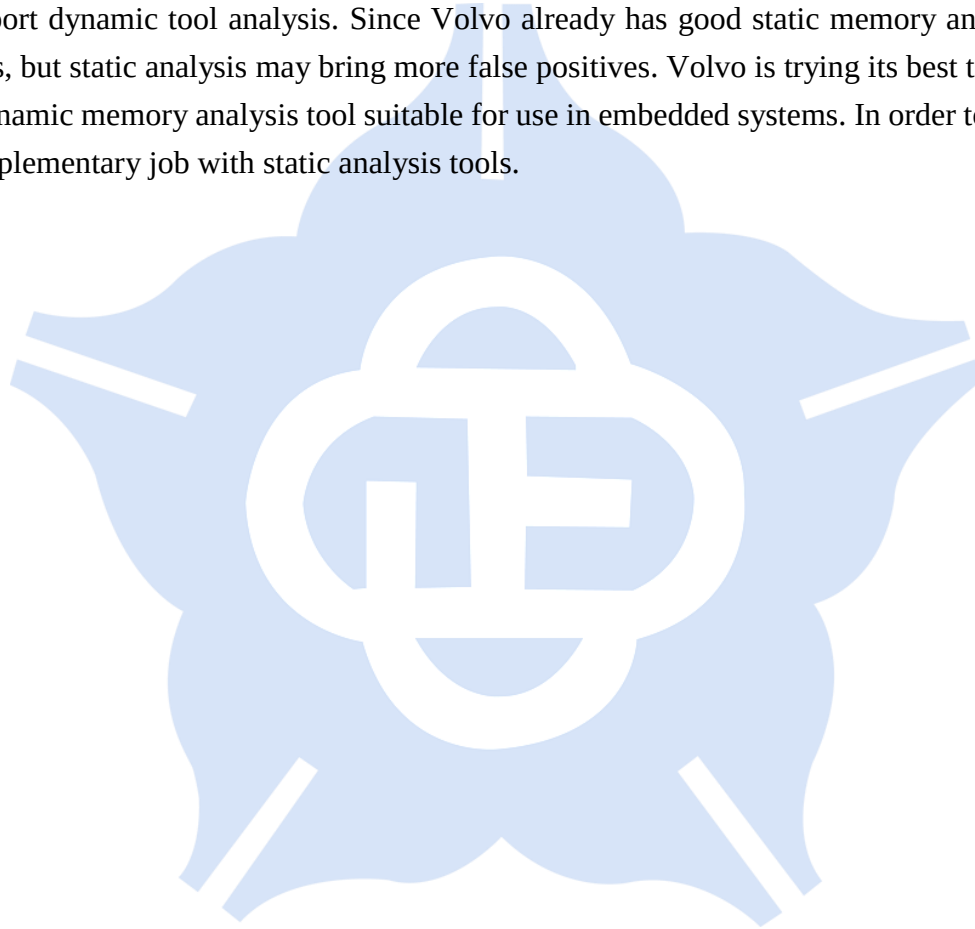
Table 4-15: Restart time of the SRA

Benchmark	Code size(bytes)	Restart time of the SRA(unit:Second)									
test	8256	3	3	3	3	3	4	4	2	2	3
basicmath_large	12656	2	2	3	3	3	2	4	4	2	3
basicmath_small	8560	4	4	4	2	3	4	3	3	3	4
isqrt	8208	3	4	3	4	2	2	2	4	3	3
rad2deg	8336	3	3	3	3	3	4	3	4	4	2
matrix2	8344	4	4	3	2	2	2	4	3	3	2



4.3. Experimental data of mtrace

We refer to the paper "Detecting Memory Errors in a Constrained Embedded Linux System"[36], which compares the current common dynamic memory analysis tools of C and C++. They cooperated with the well-known car company Volvo. Since the microcomputer control of the car is built by an embedded system, the difficulty of the embedded system is that the memory is not large enough and the hardware may not support dynamic tool analysis. Since Volvo already has good static memory analysis tools, but static analysis may bring more false positives. Volvo is trying its best to find a dynamic memory analysis tool suitable for use in embedded systems. In order to do a complementary job with static analysis tools.



First of all, we can see Table 4-16, which explains the information related to the memory error used by the author, the test number of the CWE used by the author, and the author uses Volvo developers to meet and record the most important memory errors they think, and what the author thinks Dangerous mistakes supplement the list.

Table 4-16:CWE ID.

CWE ID	Case Name
CWE-121	Stack-based Buffer Overflow
CWE-122	Heap-based Buffer Overflow
CWE-124	Stack-based Buffer Underflow
CWE-124	Heap-based Buffer Underflow
CWE-126	Stack-based Buffer Overflow
CWE-126	Heap-based Buffer Overflow
CWE-127	Stack-based Buffer Underflow
CWE-127	Heap-based Buffer Underflow
CWE-400	Stack Overflow (stack exhaustion)
CWE-401	Memory leak
CWE-415	Double free
CWE-416	Use after free
CWE-457	Use of Uninitialised Variable
CWE-468	Incorrect Pointer Scaling
CWE-476	NULL Pointer Dereference
CWE-562	Use after return
CWE-588	Attempt to Access Child of a Non-structure Pointer
CWE-590	Free of Memory not on the Heap
CWE-665	Improper Initialisation
CWE-762	Mismatched Memory Management Routines

Next, we will introduce their Overhead benchmark test options, four benchmark tests from the MiBench suite, which provides benchmark tests for embedded architectures. Simultaneously three common Linux utilities are used as benchmarks: grep, find, and gzip. In addition, three benchmarks are selected from the LLVM test suite, where two are from the MallocBench catalog, and one is from the VersaBench catalog. Table 4-17 lists all, benchmark programs.

Table 4-17: Benchmark programs for measuring overheads.

C Benchmarks:	C++ Benchmarks:
dijkstra	city
fft	employ
madplay	family
cjpeg	primes
8b10b	simul
cfrac	deriv2
espresso	life
find	shapes
grep	tramp3d-v4
gzip	

In the measurement method of execution time, they decide the real time after doing it, because task clock time only measures the time the program executes on the CPU. They believe that real time is more representative of the actual time required by the program, because real time includes any time the program spends waiting for resources.

Because this [36] paper has made a complete comparison of the common C and C++ dynamic memory analysis tools on the market, we will refer to Figure 5.1 and Figure 5.2 in [36], which represent the average of the median execution time overhead of C and C++, and also show that the execution time of mtrace in C and C++ environments is a law-abiding. Next, refer to Figure 5.3 and Figure 5.4 in [36]. They represent the average memory overhead of C and C++ in peak RSS and peak USS. It also shows that the memory usage of mtrace in C and C++ is almost negligible. Finally, refer to Table C.2 and Table C.3 in [36]. The author uses the memory error data set to test the dynamic analysis tool, the number and type of memory failures found in C and C++ by the test analysis tool. They illustrate that mtrace is in C The search for memory leaks in the environment is the same as that of Valgrind, and the search for memory leaks in the C++ environment is better than Valgrind. Because the method we proposed is to modify `mtrace.c`, but does not modify the detection method, Therefore, [36] can be used as a reference for the measurement results. Based on the above characteristics, mtrace is particularly suitable for use in embedded systems with limited resources.

Finally, we use the memory leak and double free examples in Figure 4.11, which are screenshots of memory tracking executed in DMTCP and general programs, respectively. This information will be automatically output as a log file when the user executes a specific program, as shown in Figure 4.12 and 4.13. Next, use the shell script (`glibc.sh`) we developed to help users check out memory leak and double-free errors. As shown in Figure 4.14, it also has the sort of memory leak size, memory leak and double free, and the related memory address.

```
int *p =(int *) malloc(10);
char *pw =(char *) malloc(100);
char *ps =(char *) malloc(10);
free(ps);
free(P);
free(P);
return(0);
```

Figure 4.11: Example with memory leak and double-free errors.

```

Start
cal/lib/dmtcp/libdmtcp_alloc.so:(malloc+0x27) [0x7fdd377f2237] + 0x5562bf4f1690 0x11c00 /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(calloc+0x2e) [0x7fdd377f21ae] + 0x5562bf5032a0 0x160 /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(calloc+0x2e) [0x7fdd377f21ae] + 0x5562bf503410 0x410 /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(calloc+0x2e) [0x7fdd377f21ae] + 0x5562bf503830 0x410 /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(calloc+0x2e) [0x7fdd377f21ae] + 0x5562bf503c50 0x410 /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(malloc+0x27) [0x7fdd377f2237] + 0x5562bf504070 0x400 /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(malloc+0x27) [0x7fdd377f2237] + 0x5562bf504480 0xa /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(free+0x26) [0x7fdd377f2456] - 0x5562bf504480
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(malloc+0x27) [0x7fdd377f2237] + 0x5562bf504480 0xa /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(free+0x26) [0x7fdd377f2456] - 0x5562bf504480
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(malloc+0x27) [0x7fdd377f2237] + 0x7fdd30000b60 0x18 /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(malloc+0x27) [0x7fdd377f2237] + 0x5562bf504480 0xa /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(free+0x26) [0x7fdd377f2456] - 0x5562bf504480
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(malloc+0x27) [0x7fdd377f2237] + 0x5562bf504480 0xa /usr/local/lib/dmtcp/libdmtcp_alloc.so
/usr/local/lib/dmtcp/libdmtcp_alloc.so:(free+0x26) [0x7fdd377f2456] - 0x5562bf504480

```

Figure 4.12: Screenshot of DMTCP memory tracking.

```

= Start
@ ./test1: [0x5589f457717f] + 0x5589f53a7690 0xa ./test1
@ ./test1: [0x5589f4577180] + 0x5589f53a76b0 0x64 ./test1
@ ./test1: [0x5589f4577195] + 0x5589f53a7720 0xa ./test1
@ ./test1: [0x5589f45771ab] - 0x5589f53a7720
@ ./test1: [0x5589f45771b7] - 0x5589f53a7690
@ ./test1: [0x5589f45771c3] . 0x5589f53a7690

```

Figure 4.13: Screenshot of memory tracking of general programs.

```

pi@ubuntu:~/test$ ./glibc.sh mtrace.out
8
double free : 0x5589f53a7690
memory leak : 0x5589f53a76b0
100 byte

```

Figure 4.14: Evaluation result of glibc.sh memory test.

Chapter 5 Conclusion

This paper proposes a method of combining system-level redundancy and software rejuvenation. At the system-level redundancy, a reliable structure is formed through the primary system and the redundant system. The heartbeat mechanism can check whether the primary system and the redundant system are faulty. Integrate DMTCP and GlusterFS to provide the system with checkpoints for restoring program status and reliable storage space. In addition, a software rejuvenation function is added to the primary system and the redundant system. When the user's program memory exceeds the limit, the software can be rejuvenated in real time, helping users increase the running time. We have designed two assistant tool to help users establish system-level redundancy and software rejuvenation. Finally, we modify GNU's glibc to help users instantly output memory usage. The experimental results under benchmark tests and failure conditions show that we can ensure that as long as one system is operating normally, the target program can continue to perform its work correctly. At the same time, continue to provide fault tolerance and software rejuvenation-related functions.

In the future, we will support multiple Raspberry Pi fault-tolerant services. First of all, we can choose to modify the distribution of cluster services, which can be modified so that each Raspberry Pi has the roles of the primary system and redundant system, and they monitor each other. If one of them fails, the target system can be transferred to the lower-loaded primary system to run. Another way we can choose a Raspberry Pi as the primary system to support multiple redundant systems. As for how many redundant systems are needed, it can be determined by the protection situation. For example, users can choose multiple redundant systems, fault-tolerant protection will be improved, but the cost will increase. Therefore, the issue of hardware cost and protection capabilities will be left to the user's choice.

Finally, we will consider testing our method on wheeled mobile robots or drones in the future. Two Raspberry Pi are needed to do the real-time exchange system work. Because the exchange process takes about 3 seconds, we will choose another hardware for continuous power output. After the successful exchange, Raspberry Pi will continue to protect the software systems of wheeled mobile robots or drones.

Reference

- [1] Baumann, “Soft errors in advanced computer systems,” IEEE Des. Test, vol. 22, no. 3, pp. 258–266, 2005.
- [2] R. C. Baumann, “Radiation-induced soft errors in advanced semiconductor technologies,” in IEEE Transactions on Device and Materials Reliability, vol. 5, no. 3, pp. 305 – 316, Sep. 2005.
- [3] G. A. Reis, J. Chang, N. Vachharajani, R. Rangan, and D. I. August, “Swift: Software implemented fault tolerance,” in Proceedings of the International Symposium on Code Generation and Optimization (CGO’05), Washington, DC, USA: IEEE Computer Society, pp. 243 – 254, 2005.
- [4] A. Shye, J. Blomstedt, T. Moseley, V. J. Reddi, and D. A. Connors, “Plr: A software approach to transient fault tolerance for multi-core architectures,” in IEEE Transactions on Dependable and Secure Computing, vol. 6, no. 2, p. 135– 148, Apr. 2009.
- [5] C. Wang, H.-S. Kim, Y. Wu, and V. Ying, “Compiler managed software-based redundant multi-threading for transient fault detection,” in International Symposium on Code Generation and Optimization (CGO’ 07), pp. 244 – 258, Mar. 2007.
- [6] S. K. Reinhardt and S. S. Mukherjee., “Transient fault detection via simultaneous multithreading,” in International symposium on Computer architecture), p. 25 – 36, 2000.
- [7] A. A. vizenis, “The n-version approach to fault-tolerant software,” in IEEE Transactions on Software Engineering, pp. 1491–1501, Dec. 1985
- [8] “Raspberrypi official,” Website. Online available at <https://www.raspberrypi.org>.
- [9] A. Davies and A. Orsaria, “Scale out with glusterfs,” Linux Journal, vol. 2013, no. 235, 2013.
- [10] “Glusterfs documentation,” Website. Online available at <https://docs.gluster.org>.
- [11] J. Ansel, K. Aryay, and G. Cooperman, “Dmtcp: Transparent check-pointing for cluster computations and the desktop,” in IEEE International Symposium on Parallel & Distributed Processing, pp. 1–12, May. 2009.
- [12] Marshall E. Fatal error: how patriot overlooked a scud[J]. Science, 1992, 255(5050):1347-1347.
- [13] M. Grottke, A. P. Nikora and K. S. Trivedi, "An empirical investigation of fault types in space mission system software," 2010 IEEE/IFIP International Conference on

- Dependable Systems & Networks (DSN), Chicago, IL, 2010, pp. 447-456, doi: 10.1109/DSN.2010.5544284.
- [14] Y. Huang, C. Kintala, N. Kolettis, and N. Fulton, "Software rejuvenation: analysis, module and applications," in Proc. Twenty-Fifth International Symposium on Fault-Tolerant Computing, 1995, pp. 381–390.
- [15] S. Garg, A. Puliafito, M. Telek, and K. S. Trivedi, "Analysis of software rejuvenation using Markov regenerative stochastic Petri net," in Proc. Sixth International Symposium on Software Reliability Engineering, 1995, pp. 24–27.
- [16] J. Zhao, Y. Jin, K. S. Trivedi and R. Matias Jr., "Injecting Memory Leaks to Accelerate Software Failures," 2011 IEEE 22nd International Symposium on Software Reliability Engineering, Hiroshima, 2011, pp. 260-269, doi: 10.1109/ISSRE.2011.24.
- [17] M. Yakhchi, J. Alonso, M. Fazeli, A. A. Bitaraf and A. Patoohy, "Neural network based approach for time to crash prediction to cope with software aging," in Journal of Systems Engineering and Electronics, vol. 26, no. 2, pp. 407-414, April 2015, doi: 10.1109/JSEE.2015.00047.
- [18] R. Matias and P. J. F. Filho, "An Experimental Study on Software Aging and Rejuvenation in Web Servers," 30th Annual International Computer Software and Applications Conference (COMPSAC'06), Chicago, IL, 2006, pp. 189-196, doi: 10.1109/COMPSAC.2006.25.
- [19] V. Castelli *et al.*, "Proactive management of software aging," in *IBM Journal of Research and Development*, vol. 45, no. 2, pp. 311-332, March 2001, doi: 10.1147/rd.452.0311.
- [20] D. J. Scales, M. Nelson, and G. Venkitachalam, "The design of a practical system for fault-tolerant virtual machines," in ACM SIGOPS Oper.Syst. Rev., pp. 30–39, Dec. 2010.
- [21] "Leosys," Website, 2015. Online available at <http://www.leosys.com/index.php/aboutleo/newsletter/itdevelopment/> 10-2015-02-25-02-29-10.
- [22] W.-C. Liao, "Implementing a virtual machine based fault tolerance system," Master's thesis, National Taiwan University, 2017.
- [23] J.-W. Shang, "Using non-deterministic event log and replay to support virtual machine fault tolerance of kernel-based virtual machine," Master's thesis, National Central University, 2016.
- [24] "Why should you use Valgrind?" Website, 2021. Online available at <https://valgrind.org/info/about.html>

- [25] “Valgrind Memcheck 源碼分析” Website, 2021. Online available at <https://blog.csdn.net/Linsoft1994/article/details/79562518>.
- [26] “What is glibc ?” Website, 2021. Online available at <https://www.gnu.org/software/libc/>
- [27] J. S. Plank, M. Beck, G. Kingsley, and K. Li in Libckpt: transparent checkpointing under Unix, pp. 18–18, Jan, 1995.
- [28] M. Litzkow, T. Tannenbaum, J. Basney, and M. Livny in Checkpoint and migration of UNIX processes in the Condor distributed processing system, 1997.
- [29] “Make your projects more reliable,” John Shovic, 2015. Online available at <http://www.raspberry-pi-geek.com/Archive/2015/09/Making-your-projects-more-reliable>.
- [30] “stress-ng” Online available at <https://wiki.ubuntu.com/Kernel/Reference/stress-ng>
- [31] “A stress-testing Swiss army knife” Online available at <https://elinux.org/images/5/5c/Lyon-stress-ng-presentation-oct-2019.pdf>
- [32] L. M. Silva, J. Alonso and J. Torres, "Using Virtualization to Improve Software Rejuvenation," in IEEE Transactions on Computers, vol. 58, no. 11, pp. 1525-1538, Nov. 2009, doi: 10.1109/TC.2009.119.
- [33] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, “Mibench: A free, commercially representative embedded benchmark suite,” in Proceedings of the Workload Characterization, pp. 3 – 14, Dec. 2001.
- [34] V. živojnović, J. M. Velarde, C. Schläger, and H. Meyr, “DSPSTONE: A DSP-oriented benchmarking methodology,” in Proceedings of the International Conference on Signal Processing and Technology (ICSPAT’94), 1994.
- [35] M. Yakhchi, J. Alonso, M. Fazeli, A. A. Bitaraf and A. Patoohy, "Neural network based approach for time to crash prediction to cope with software aging," in Journal of Systems Engineering and Electronics, vol. 26, no. 2, pp. 407-414, April 2015, doi: 10.1109/JSEE.2015.00047.
- [36] Nyberg, Fredrik, and Emil Bengtsson. "Detecting Memory Errors in a Constrained Embedded Linux System." LU-CS-EX (2020).